

Ponens: An Open-Source Framework for Verifiable, Auditable, and Composable Agent Reasoning

The Reasoning Trace as a Defeasible, Proof-Carrying, Composable Context

Denis Ignatovich¹

denis@imandra.ai

Grant Passmore^{1,2}

grant@imandra.ai

Abstract

Agentic AI produces reasoning at scale, but that reasoning is ephemeral, non-reproducible, and — most consequentially — *not composable*: there is no account of what a reasoning result depends on, so no principled way to say what a later change invalidates. Audit logs record *what happened*; they are not reasoning *states*. We present **Ponens**, an open-source framework that makes an agent’s reasoning trace a first-class logical object: a **defeasible, proof-carrying context** Γ in which every claim carries a certificate — which oracle produced it, under what assumptions, whether it is *fresh*, and whether it is *contested*. The oracle that produced a claim is a *generic type* — a prover, a test runner, a static analyzer, a database queried for a fact, an LLM judge, a human sign-off — and every result carries the honest *strength* of its guarantee (proof > sat > tests > static analysis > attested), so a goal reports the weakest link it rests on; a requirement may point at a *reference* model outside the repository — a regulation, a venue specification — and evidence against it goes stale when the reference moves. The trace is an append-only *state* with well-formedness invariants, simultaneously *memory*, a *governance surface*, and a *logical context*. Over it an *evidence logic* judges an agent’s work on two axes — **goals** (the definition of done, as outcome properties of what was established) and **policies** (temporal constraints on how it was established, run by an LTL-style model checker over the action timeline) — both resolved against the trace rather than self-reported. Because the trace is content-addressed, its verdicts are re-checkable by any party and any edit is tamper-evident; cryptographic sign-offs over the content hash (SSH, GPG, or keyless sigstore), each bearing the signer’s *role* and *disposition*, turn it into a non-repudiable **audit trail** in which the party that did the work does not grade it. An important technical contribution is **composition over traces**: a merge is the composition operator of the evidence logic, and we show how to decide *soundly* what a merge invalidates versus what it provably preserves — the moment at which two individually-valid results can become jointly invalid. We give a durable notion of component identity that lets evidence bind to code across renames and moves. The *entire framework* is given a formal model — a single state-transition machine in which the trace is the state — and machine-checked in ImandraX³: every invariant, both evidence-logic axes, and the merge composition are discharged over that one state, the system verifying its own logic. The implementation is reasoner-agnostic at its core, with a concrete instantiation over IML/ImandraX and a `git` post-merge trigger that reports, on a real merge, exactly which established results the merge put at risk. Finally we argue the same construction underpins *multi-reasoner* composition: a cross-branch dependency and a cross-reasoner dependency are the same object — an explicit, tracked assumption.

¹Imandra, Inc.

²University of Cambridge

³ImandraX is Imandra’s automated reasoning engine: <https://www.imandrax.dev>.

Contents

I	The reasoning trace	3
1	Introduction	4
1.1	Motivating examples	4
1.2	Overview: an evidence logic over the trace	6
2	The reasoning trace as state and context	6
2.1	State, operations, and invariants	6
2.2	A context is more than a log	7
3	The trace format	7
3.1	Canonical model and interchange projection	8
3.2	Actions: the ground-truth log	8
3.3	Artifacts and lineage	9
3.4	Formal results	10
3.5	The residual surface: the negative space	10
3.6	Carried-forward: the positive dual	11
3.7	Reproducibility and execution environments	11
3.8	A worked trace	12
4	Oracles and evidence strength	14
4.1	Mechanism and guarantee	14
4.2	Attribution and the honesty rule	14
4.3	The evidence fingerprint and the Observation	15
4.4	Consequences for the evidence logic	16
5	Freshness	16
5.1	The fingerprint	16
5.2	The derived verdict	17
5.3	Reference freshness	17
5.4	The formal model	18
II	Evidence logic and composition	18
6	Goals	19
6.1	Constructing a goal	19
6.2	Three orthogonal axes	20
6.3	The acceptance criterion and the property language	20
6.4	Resolution by lineage	21
6.5	Requirements against a reference	23
6.6	Faithfulness — the certified axis	24
6.7	Authorable today versus open	24

7	Policies	25
7.1	The policy object	25
7.2	The formula language	25
7.3	Evaluation semantics	28
7.4	Policy sources and packs	28
7.5	Compliance packs: one substrate, many regimes	29
7.6	Worked examples	30
7.7	Met versus governed	32
8	Composition	32
8.1	Composition over traces	33
8.2	Durable component identity	34
8.3	Generalization: multi-reasoner composition	35
9	Formal model and verification	35
III	The system	38
10	The Ponens system	38
10.1	The CLI in practice	38
10.2	The LTL policy checker	40
10.3	The record as it is read	41
10.4	Continuous integration: the Action	41
11	Trust, audit, and review	42
11.1	Audit readiness: a mechanical floor	42
11.2	Signatures and non-repudiation	43
11.3	Review and handoff	43
11.4	The audit process	44
12	Interchange and sync	45
12.1	Interchange: projecting to W3C PROV	45
12.2	Sync and binding	46
IV	Demonstration and discussion	47
13	Worked examples	47
13.1	A verified pricing change through a merge	47
13.2	Governing an agentic trading workflow (FIX)	48
14	Related work	49
15	Conclusion	50

Part I

The reasoning trace

1 Introduction

An agent that writes code, proves a property, generates tests, or discharges a compliance obligation produces *reasoning*. Today that reasoning is treated as exhaust: it scrolls past in a transcript, or is flattened into an audit log, and is gone. Two problems follow. First, it is not *reproducible* — a reader cannot re-derive the result. Second, and more fundamentally, it is not *composable*: nothing records what a result depended on, so when the code changes — an edit, a pull, a branch merge — there is no principled way to determine which prior conclusions still hold. Testing and review sample the space; they cannot certify that a change left a body of reasoning intact, and they are structurally blind to the case where two *individually valid* changes are *jointly* invalid. This paper presents the foundations of *Ponens*, an open-source framework⁴ in which an agent’s reasoning is not exhaust but a first-class, composable, verifiable record.

Our thesis is that the right object is not a log but a **state**: the reasoning trace should be a first-class logical context that the system computes *over*, in which every recorded claim carries the warrant that makes it usable as a premise. We make this precise and show it is not merely an accounting convenience: it is what makes reasoning *compose*.

1.1 Motivating examples

Six failures, all but invisible to testing and review as practiced today, motivate the construction. Each is a case where individually-reasonable work yields a silently-wrong conclusion because nothing recorded what that conclusion *depended on*.

1. The merge that silently breaks a proof. On one branch an agent proves that **charge** never double-charges on a retry. On another, a colleague refactors **apply_discount** — a helper in that proof’s dependency closure — to change rounding. Both branches pass their own tests and reviews; each result is valid in isolation. The merge yields code in which the proof no longer holds, yet CI stays green because no test exercises the retry-and-discount interaction. The reasoning was *individually valid but jointly invalid*, and nothing raised a signal at the merge. Deciding *soundly* which established results a merge preserves and which it puts at risk is composition over traces (§8.1).

2. Stale evidence that still reads as green. A verification result from last week sits in the record with a green checkmark. Since then a function in its dependency closure changed, but nothing re-examined the proof — so the dashboard still shows it passing: evidence that is stale yet presented as current. Freshness must be a *derived* verdict recomputed from the current code, never a stored flag (§2, §3); a claim resting on stale evidence is not “done.”

3. “Done” that was never established. An agent writes “I verified the balance invariant holds” in its summary. There is no proof, no test — only prose — and a reviewer reading a plausible transcript takes it at face value. The gap between what was *asserted* and what was

⁴*ponens* is open source: <https://www.ponens.dev> (documentation and the full trace, goal, and policy specifications) and <https://github.com/imandra-ai/ponens> (source).

established is where most agent-work failures hide. A goal resolves only against a checkable artifact in the component’s lineage, never from prose (§6); an unbacked “verified” is recorded as **Unverified**.

4. A rename that orphans the evidence. A refactor renames `charge` to `charge_customer`. Every proof, test, and decomposition about it is now attached to a name that no longer exists; naively, the evidence is lost and the “new” function looks entirely unverified. Durable component identity (§8.2) binds evidence to a code element across renames and moves, so a rename is one component, not a delete plus an add.

5. Compliant only in hindsight. A change ships correct but was produced in a way that violates a required process — committed before the tests ran, or merged without the independent review a regulated codebase demands. The artifact is right; *how* it was produced is not, and a flat log of what happened cannot answer a temporal question about ordering. Policies (§7) are temporal constraints on the action timeline, evaluated against the trace, so “done” and “done in a compliant way” are separate, checkable verdicts.

6. The rule that moved. A payment service was checked, thoroughly, against the provider’s specification of its `PaymentIntent` lifecycle; a reporting pipeline was checked against an article of the regulation it implements. Then the provider re-issued the specification, or the regulator amended the article. No file in the repository changed, so no test, no diff and no freshness check keyed on the code notices; every “conforms” still reads green against a text that no longer says what it said. A requirement that points at a model *outside* the repository must record what it was judged against (§6.5), and the reference must be a dependency like any other, so that a revision to it stales the evidence (§5.3).

None of these is exotic; together they are the ordinary texture of multi-agent, multi-branch development. What they share is a single missing ingredient — a record of *what each conclusion depended on*, kept in a form the system can compute over. That record is the reasoning trace.

Contributions.

- A formal model of the reasoning trace as state (§2, §3) and the argument that it is a *defeasible, proof-carrying context*, with three properties — non-monotonic, query-carrying, and well-founded — that make it a *good* context (§2.2).
- **Graded evidence from any oracle** (§4): the oracle as a generic type with an honest strength ladder and a subject fingerprint, so a proof, a test run, a database lookup and a sign-off are governed evidence on one footing; and **requirements against a reference** (§6.5), the binding of a repository to a model outside it, with reference freshness (§5.3).
- **An evidence logic in two axes:** *goals* (§6) — a property language of typed, lineage-resolved acceptance criteria over code components — and *policies* (§7) — a first-order temporal language ($G/F/U/S$) over the action timeline; both resolved against the trace, with worked examples.
- **Composition over traces** (§8.1): a merge as the composition operator of the evidence logic, with a sound *affected-set* decision (carry forward the provably-unaffected, flag the rest) and a checkable *totality* invariant.

- **Durable component identity** (§8.2): binding evidence to code across rename/move so composition survives refactoring, with a never-conflate resolver.
- **A formal model and its verification** (§9): the framework modelled as a single state-transition machine (the trace as state), its entire invariant set proved by the system on itself with ImandraX — 195 proof obligations over one state, 0 failures.
- The system (§10) — the CLI, the policy checker, the one overview every screen reads and the continuous-integration action — its trust, audit and review surfaces (§11), its interchange and sync (§12), and the *multi-reasoner* generalization (§8.3).

1.2 Overview: an evidence logic over the trace

An agent’s work is judged over the trace by an *evidence logic* with two axes. **Goals** state *what* had to be established — outcome properties, resolved against the evidence recorded in the trace (§6); **policies** state *how* the work had to be produced — temporal constraints over the timeline of actions (§7). These are one *two-sorted* logic: the same connectives and quantifiers range over an *artifact* sort (for goals) or an *action* sort (for policies). Their verdicts — **met** and **governed** — are *orthogonal by construction* (how the work was done cannot change what was established), a property we machine-check (§9). A third axis, **certified** — a sign-off by a party *other than* the producer — rides the same structure, which is what lets the trace be verified across parties. Because every claim carries its warrant, all three axes are decided by *re-checking the trace*, never by trusting a report; and at a merge, composition (§8.1) re-evaluates them all.

2 The reasoning trace as state and context

2.1 State, operations, and invariants

The state σ is an **append-only** record with five parts: an ordered log of *actions*; a lineage DAG of typed *artifacts* (the evidence — models, proofs, decompositions, tests, edits), each produced by an action and linked to its inputs by **derived_from**; a *goal* lens (the definition of done); a *residual surface* (the negative space — assumptions, gaps, counter-evidence); and a *derived* freshness verdict over each result. Two kinds of operation act on σ :

$$\text{extend}(\sigma, op) \rightarrow \sigma' \sqsupseteq \sigma \quad \text{query}(\sigma, op) \rightarrow \text{answer} \quad (\sigma \text{ unchanged}).$$

extend appends established evidence, declares a residual, or sets the goal; **query** resolves a goal, checks a policy, or renders a view. Freshness is never stored — it is recomputed from the current code, so the state cannot silently disagree with reality.

Two logics. The trace separates the *object logic* — where a domain’s behaviour is analysed and results are produced (in our instantiation, IML discharged by the ImandraX engine [1]) — from the *evidence logic*: properties *of the trace itself* (“there exists a fresh, uncontested proof, rooting in this component, that the property holds”). A goal is a formula in the evidence logic; its atoms (**proved**(c), **fresh**(e), **conforms**(S), ...) quantify over artifacts, lineage, and freshness. This separation is what lets one substrate host many object-level reasoners (§8.3).

Invariants. The state maintains five invariants, on which everything downstream rests:

I1 Append-only. Evidence is immutable; re-analysis *supersedes* (a new revision), never mutates.

I2 Lineage-closed. Every `derived_from` edge points at a strictly *earlier* result — the acyclicity condition that forbids circular premises.

I3 Evidence-grounded. Resolution reads only from evidence in lineage, never from prose.

I4 Freshness-sound. A result is **fresh** iff its target’s dependency-closure content is unchanged.

I5 Conditional growth. Re-establishing a fresh result is a no-op; only new work appends.

These are not asserted informally: §9 discharges them as machine-checked theorems.

2.2 A context is more than a log

An audit log is written *after* the fact and read by humans. A reasoning context is computed *over*. The difference is that every recorded claim in σ carries a **certificate**: which oracle produced it (a sound engine versus a heuristic model), under what *assumptions*, whether it is *fresh*, and whether it is *contested* by an open defeater. Reading σ is therefore reading a body of evidence with its warrant attached — exactly the metadata a downstream step needs to decide whether a prior result is an admissible premise. The trace serves as a context in two senses: an *evidence-logic* context (the model that goal resolution and policy checking evaluate against), and an *object-logic* context (an artifact can be *projected* into a downstream reasoner’s logic as a premise, carrying its validity conditions as guards).

Three properties make it a *good* context, and each is one of the invariants of §2:

- **Non-monotonic** (I4). Freshness decays and defeaters retract, so *adding* evidence can *withdraw* a prior conclusion. A proof that was sound goes `at_risk` when its source drifts. This is the correct model of a living codebase — and it matches the semantics of the non-monotonic reasoners (defeasible logic, answer-set programming, the event calculus) one might plug in at the object level.
- **Carries the query side.** Residuals are *open goals*: the negative space (what is not yet established) is first-class alongside the positive. A context that carries its own open questions is directly usable by goal-directed and abductive reasoners.
- **Well-founded by construction** (I2). Lineage acyclicity means no claim can rest, transitively, on itself: the context can always be evaluated bottom-up.

So one append-only structure is at once **memory** (what was done), a **governance surface** (what policy checks), and a **logical context** (what may be assumed, and under what warrant). The rest of the paper is what this buys: composability.

3 The trace format

Section 2 gave the trace as an abstract state σ : an append-only log, a lineage DAG of typed artifacts, a goal lens, a residual surface, and a derived freshness verdict. This section makes that model concrete. The format is the load-bearing artifact of the whole construction: the invariants of §2, the certificate of §2.2, and the composition of §8.1 are all statements *about the typed objects defined here*. We present the canonical types faithfully but distilled, and close with a worked interchange trace that exhibits every discriminator the rest of the paper relies on.

3.1 Canonical model and interchange projection

The specification is layered. The **canonical model** is the semantic source of truth: strongly typed, algebraic where a set of cases is closed, and the model against which invariants and semantics are defined. The **interchange model** is a JSON projection of it, for persistence, transport, and cross-language interoperability. The design rule is one-directional — *specify the strongest semantic model first; derive the wire format from it, not the reverse* — so the JSON is never authoritative. This matters for verifiability: the invariants of §9 are proved over the canonical types, and the wire format inherits them only because it is a projection.

The canonical trace is a typed record whose fields are exactly the five parts of §2 plus reproducibility and governance surfaces: an ordered **actions** list, a **meta_actions** overlay, an **artifacts** lineage DAG, **policies/policy_evaluations**, **execution_environments**, **goals**, and inter-trace **trace_links**. Every algebraic variant projects to JSON under an **explicit discriminator**: artifacts carry **artifact_type**, actions carry **category** and **type**, and verification-result variants carry tags such as **proved/refuted/sat/unknown**. Decoding must reconstruct the strict model and *reject* an invalid discriminator/payload combination — the wire boundary is where ill-typed traces are caught, not admitted.

3.2 Actions: the ground-truth log

Actions are the ordered, atomic steps — one entry per tool call — and they do not exchange free-form names; they exchange *artifact identities* via **inputs** and **outputs**. Every action shares a common record (Figure 1).

	id	: Int,
	label	: String,
	rationale	: String,
	detail	: String [?] ,
	inputs	: ArtifactId*,
action_common ::=	(outputs	: ArtifactId*,
	evidence	: Evidence*,
	observations	: Observation*,
	execution	: ExecutionMetadata [?] ,
	reproducibility	: ActionReproducibility [?] ,
	meta_action_id	: String [?]

Figure 1: The common record shared by every action: identity, labels, the **inputs/outputs** artifact-id edges, evidence and observations, and optional execution and reproducibility metadata.

The strict action type partitions steps into four **categories**, each with its own closed set of types: *activity* (**ReadFile**, **EditFile**, **RunTests**, **GitCommit**, **FormulatePlan**, ...), *gateway* (decisions, splits, loops — carrying the options and the reason each was chosen or rejected), *reasoning* (**Formalize**, **DefineVerificationGoal**, **Verify**, **StateSpaceAnalysis**, **ConformanceCheck**, **CoSimulate**, **GenerateTests**), and *governance* (**EvaluatePolicy**, **RequestApproval**, **RecordAudit**, **LinkTrace**, ...). The category/type split is the semantic layer; the concrete engine is recorded separately in **execution_metadata** (**tool** = "imandrax",

`method_ = "region_decomposition"`, a `determinism` tag), which keeps the format *reasoner-agnostic* — the same `StateSpaceAnalysis` action denotes the same reasoning operation whether ImandraX or another engine realized it.

Over this atomic log sits an optional **meta-action** overlay: an ordered, non-overlapping grouping of actions into units of *intent* (a goal-to-steps hierarchy via `parent_id`), carrying a `title`, `intent`, `outcome`, and a `source` on a fidelity ladder (`PlanDeclared` > `TurnSegmented` > `IntentInferred`). This is the *curated narrative* of a session: the atomic actions remain the evidence, and the overlay is a producer’s claim about how they group — reviewable as a unit before drilling in.

3.3 Artifacts and lineage

Artifacts are the evidence. Every artifact carries a common record (Figure 2) and then, for the structured kinds, a typed payload aligned with its constructor.

```

                                artifact_id      : String,
                                artifact_role     : ArtifactRole?,
                                name              : String?,
                                format            : String?,
                                revision          : Int?,
artifact_common ::= ( producer_action_id : Int?,
                        derived_from      : ArtifactId*,
                        supersedes        : String?,
                        content_ref       : String?,
                        summary           : String?,
                        component_id      : String?,
                        metadata          : ArtifactMetadata? )

```

Figure 2: The common record shared by every artifact: the record identity `artifact_id`, semantic role, the `derived_from` lineage edges, the `supersedes` revision link, and the rename-stable `component_id`.

The artifact type is a **strict union** so that payload presence is aligned with kind and impossible combinations are unrepresentable: source code, plans, diffs, and commits carry only the common record; the reasoning kinds — `FormalizationArtifact`, `FormalModelArtifact`, `VerificationGoalArtifact`, `VerificationResultArtifact`, `StateSpaceAnalysisResultArtifact`, `ConformanceResultArtifact`, `CoSimulationResultArtifact`, `GeneratedTestsArtifact` — pair the common record with a typed payload. Two members are newer additions the rest of the paper leans on: the residual surface is folded in as a `Residual` artifact (§3.5), and **1.10 adds** `CarriedForwardArtifact` — the positive record of a merge skip (§3.6).

`derived_from` is the lineage DAG: each edge names an immediate input, so an artifact’s **dependency closure** is the transitive back-walk. This is what invariant I2 of §2 constrains — an edge points only at a strictly earlier artifact, forbidding circular premises — and what goal resolution and freshness walk. **Reversioning** is append-only (I1): an artifact is immutable once produced, so re-analysis *supersedes* (a new `revision`, linked back by `supersedes`) rather than mutating in place. “The latest result” is thus a derived query over the supersession chain, never an overwrite — which

is exactly what lets an auditor see the whole history and what lets freshness be recomputed against any point.

3.4 Formal results

The reasoning payloads carry the object-logic verdicts. A `VerificationGoal` states what is to be established (`kind` in `verify/instance/theorem/lemma/axiom`, a `target_symbol`, a list of `property_items`); a `VerificationResult` carries the outcome as a tagged variant — `ProvedResult` (with the pretty-printed proof), `RefutedResult` (with a counterexample), `SatResult`, or `UnknownResult`. The state-space, conformance, and co-simulation payloads are structurally analogous verdicts over their own object logics. Each result may additionally carry a `reasoning_fingerprint`, the anchor for freshness treated in its own right in §5.

3.5 The residual surface: the negative space

A trace records what was established — the positive space — but review and agent-to-agent handoff need the *negative* space: what was *not* established. The residual surface is the uniform, typed, queryable record of it, and **a residual is an artifact** (`artifact_type` = "Residual"): it carries its fields in `payload` and *anchors into the lineage DAG* through `derived_from`, so a gap hangs off exactly the artifact it qualifies. Positive and negative space share one addressable model.

<code>residual_kind</code>	::=	<code>Assumption</code> <code>Unverified</code> <code>OutOfScope</code> <code>Limitation</code> <code>OpenQuestion</code> <code>Defeater</code> <code>NeedsRereasoning</code> <code>CoverageRegression</code>
<code>Assumption</code>		<i>a premise relied on but not established</i>
<code>Unverified</code>		<i>something done but not checked or proved</i>
<code>OutOfScope</code>		<i>deliberately not addressed</i>
<code>Limitation</code>		<i>a constraint under which the results hold</i>
<code>OpenQuestion</code>		<i>a decision deferred to review</i>
<code>Defeater</code>		<i>counter-evidence against a claim (not a gap); carries a <code>defeater_kind</code></i>
<code>NeedsRereasoning</code>		<i>a result a merge disturbed; re-establish against the merged code</i>
<code>CoverageRegression</code>		<i>a goal whose scope gained an unproven member after a merge</i>
<code>defeater_kind</code>	::=	<code>Rebuts</code> <code>Undermines</code> <code>Undercuts</code>
<code>Rebuts</code>		<i>the claim may be false — e.g. a counterexample to a proved property</i>
<code>Undermines</code>		<i>the evidence is invalid — e.g. the model does not match the code</i>
<code>Undercuts</code>		<i>the inference is deficient — e.g. “passing tests do not establish this”</i>

Figure 3: The residual taxonomy (Pollock / SEI eliminative-argumentation): `residual_kind` enumerates the gap and counter-evidence kinds (including the merge-specific `NeedsRereasoning` and `CoverageRegression`), and `defeater_kind` classifies a defeater as a rebut, undermine, or undercut.

The first five kinds record *missing positive space* — something not established. A `Defeater` is different in kind: it records *negative evidence*, a concrete reason to believe a stated result is *wrong*, and it anchors (via `derived_from/target`) to the claim it contests while citing the counter-evidence in `related_artifact_ids`. An *open* defeater makes its target **contested** — a `Property` acceptance item over a contested proof resolves `AcceptBlocked`, not `AcceptDone` (this is

the non-monotonic retraction of §2.2, made first-class and reviewer-raisable). **1.10 adds** two merge-specific kinds: **NeedsRereasoning** (an established result whose evidence a merge disturbed) and **CoverageRegression** (a goal whose scope gained an unproven member) — the negative half of the composition account of §8.1. Each residual also carries a **severity** (impact-if-wrong, not probability), a **source** on a trust ladder (**AgentDeclared**, **PolicyDerived**, **ToolInferred**, **ReviewerAdded**), a **status** (**Open**/**Acknowledged**/**Addressed**/**Waived**), and an optional **suggested_check** that turns a warning into a work-item. Because it is an ordinary artifact, every severity/status/source field and every policy applies to a defeater unchanged.

3.6 Carried-forward: the positive dual

Composition (§8.1) needs a symmetric record on the positive side. **1.10 adds** the **CarriedForwardArtifact**: where a residual records a gap, a **CarriedForward** records a claim that was *deliberately not re-checked because it is provably safe*. It carries a falsifiable witness so an auditor can re-verify the skip (Figure 4).

```

carried_forward_basis ::= ClosureDisjoint | UninterpretedOpaque

ClosureDisjoint      the result's dependency closure did not intersect the merge's change set
UninterpretedOpaque  every touched dependency was assumed uninterpreted

carried_forward_payload ::= (
    result_id      : String,
    basis          : carried_forward_basis,
    closure        : ComponentId*,
    via_assumptions : String*
)
```

Figure 4: The **CarriedForward** record documenting a provably safe merge skip: the **basis** (disjoint closure or uninterpreted dependency) and the **closure** of component ids that serves as the auditor’s falsifiable witness.

The merged trace’s account of every prior result is thus exactly one of three under the **totality** invariant of §8.1: **CarriedForward**, **NeedsRereasoning**, or freshly re-established. The two-parent **merge_event** that occasions this is recorded on the merged trace as provenance kept *off* the artifact DAG — so I2 acyclicity (a **derived_from** edge points only at an earlier artifact) survives composition. The change set is matched by **component_id**, so a rename is one component rather than a delete-plus-add; detail is deferred to §8.1.

3.7 Reproducibility and execution environments

Reproducibility is recorded at two granularities. At the **trace** level, a **trace_reproducibility** record states a **status** (**NotReproducible**/**PartiallyReproducible**/**Reproducible**), entrypoints, and the required artifacts and environments. At the **action** level, each consequential action may carry a **reproduction_kind** (deterministic replay, tool re-execution, procedural or manual replay), a concrete **procedure** (command, working directory, arguments, or steps), and an **expected_output**. Both refer to named **execution_environments** — a toolchain, container, service, or external system with its versioned components — so a reader can reconstruct *where* a result was produced, not merely that it was. The discipline is that a consequential reasoning outcome is either directly reproducible or linked to a reproducible downstream validation.

3.8 A worked trace

Figure 5 shows the interchange projection of a minimal but complete slice: one reasoning action, the `VerificationGoal` it consumed and the `VerificationResult` it produced (with a `reasoning_fingerprint`), and one `Residual` contesting that result. It exhibits the `category/artifact_type` discriminators and the `derived_from` lineage that carries the whole construction.

```

{
  "actions": [
    { "id": 4, "category": "reasoning", "type": "Verify",
      "label": "Verify amount_inv on the payment model",
      "rationale": "capture must never exceed the authorized amount",
      "inputs": ["a2", "g1"], "outputs": ["vr1"],
      "execution": { "tool": "imandrax", "version": "2026.04",
        "method": "model_check", "determinism": "deterministic" } }
  ],
  "artifacts": [
    { "artifact_id": "g1", "artifact_type": "VerificationGoal",
      "producer_action_id": 3, "derived_from": ["a2"],
      "component_id": "cmp:amount_inv",
      "payload": { "goal_id": 1, "kind": "VerifyGoal",
        "description": "amount_captured <= amount",
        "target_artifact_id": "a2", "target_symbol": "amount_inv",
        "properties": [ { "name": "amount_inv", "status": "Pending",
          "src": "amount_captured <= amount" } ] } },

    { "artifact_id": "vr1", "artifact_type": "VerificationResult",
      "artifact_role": "proof", "producer_action_id": 4,
      "derived_from": ["g1", "a2"], "component_id": "cmp:amount_inv",
      "summary": "amount_inv proved",
      "payload": { "goal_id": 1, "goal_artifact_id": "g1", "status": "proved",
        "engine": "imandrax",
        "result": { "proved": { "proof_pp": "...", "properties": [] } },
        "fingerprint": {
          "task_checksum": "sha256:7f3c...e1", /* target + closure */
          "task_shape": "sha256:0a9b...",
          "target_symbol": "amount_inv",
          "engine": "imandrax", "engine_version": "2026.04",
          "model_artifact_id": "a2", "model_revision": 1 } } },

    { "artifact_id": "r1", "artifact_type": "Residual",
      "derived_from": ["vr1", "a13"], /* anchors to the claim it contests */
      "summary": "amount_inv refuted under interleaved capture",
      "payload": { "kind": "defeater", "defeater_kind": "rebuts",
        "statement": "capture can exceed amount under concurrent capture/refund",
        "severity": "critical",
        "target": { "target_type": "artifact", "target_id": "vr1" },
        "related_artifact_ids": ["a13"], /* the counterexample */
        "source": "reviewer_added", "status": "open" } }
  ]
}

```

Figure 5: A worked interchange trace: a Verify reasoning action, the VerificationGoal g1 and proved VerificationResult vr1 (with its reasoning_fingerprint), and an open Residual defeater r1 that rebuts vr1 — exhibiting the discriminators and derived_from lineage the rest of the paper relies on.

Reading it as a context: vr1 is a *proved* result, and its fingerprint lets a consumer recompute whether it is still *Fresh* against the current model. But r1 is an *open* Defeater that *Rebuts* it (anchored by *derived_from* and *target*), so the claim is *contested*: a *Property* obligation over vr1 resolves *AcceptBlocked*, not *AcceptDone*. The certificate travels with the claim — which oracle (imandrax), under what task (the fingerprinted closure), whether fresh, whether contested — which

is precisely what the freshness of §5 and the composition of §8.1 compute over.

4 Oracles and evidence strength

Every certificate in σ begins with *who produced the claim*. §2.2 named that party an *oracle* and §3 recorded its verdicts; this section makes the oracle a type. The point is not taxonomy. A proof, a test run, a static analysis, a database queried for a fact, an LLM’s judgement and a human sign-off are all evidence an agent’s work rests on, and a goal that rests on several of them is exactly as strong as the weakest. Unless each result carries the *honest* strength of its own guarantee, “proved” silently absorbs “a source said so,” and the evidence logic grades a chain by its strongest link. The construction is therefore *reasoner-agnostic* one level down — ImandraX is one prover among several — and *oracle-agnostic* one level up: a reasoner is one kind of oracle among several, and goals, policies and composition treat them uniformly as attributed, graded evidence.

4.1 Mechanism and guarantee

Two classifiers travel with every result, and they are orthogonal. The **mechanism**, `oracle_type`, says *how* the evidence was obtained (Table 1). The set is *open*: the standard names are a classification, a consumer warns on an unknown one and never rejects it, and a specialized oracle names the standard type it **specializes** so that filters and policies written against the standard name still match. The **guarantee**, `evidence_strength`, says *what the output warrants*, in a total order, strongest first:

proof > sat > tests > static_analysis > attested.

A **proof** holds over the entire, possibly unbounded, state space; **sat** is a witness or a bounded check; **tests** is empirical evidence from executed cases; **static_analysis** is a sound-ish over-approximation without execution; **attested** is asserted by a source, a judge or a person and not mechanically established. The two are deliberately distinct: a reasoner running as a model checker yields **sat**, not **proof**; a monitor never yields more than **attested**, however authoritative its source. There is no separate **observed** grade: an observation *is attested*, and the authority of its source is a matter of attribution (§4.2), not of strength. Evidence that omits a strength is unranked and sorts last. The order is locked and propagates everywhere strength is read — goal resolution, policy predicates, composition and every rendered screen.

4.2 Attribution and the honesty rule

Every evidence artifact an oracle returns carries an **attribution block** in its payload: `payload.oracle = { id, oracle_type, evidence_strength, version }` — the registry id of the oracle, its mechanism, the strength of *this* result, and the oracle version that produced it. Attribution is what makes a claim re-checkable by a party who was not there: a reviewer sees that a **VerificationResult** came from ImandraX at a given version, that an **Observation** came from the settlement-calendar database at schema version seven, that an **AnalysisNote** was an LLM judge’s opinion — and can re-ask the same oracle, or a different one of the same type. Reasoning results keep their `engine / engine_version` fields; for a reasoner they equal the oracle’s `id / version`, and a consumer reading an older trace derives the block from them, so attribution is additive.

The **honesty rule** closes the gap between capability and outcome. The strength on a *result* reflects the actual verdict, never what the oracle *could* have produced: **proved** or **refuted** carry

oracle_type	What it does	Instances
reasoner	formal engines: proof and decision procedures	ImandraX, Lean, Z3, a model checker
tester	dynamic execution: run it and observe	test runners, property-based testing, fuzzers, simulators, backtests
analyzer	static inspection without execution	type checkers, SAST, linters, a schema or contract check
monitor	observational evidence from a live system or an external source	a database or reference-data store queried for a fact, telemetry, a rate-fixing, calendar or rulebook-version lookup
judge	heuristic or probabilistic assessment	an LLM as judge, rubric evaluators, model-graded evaluation
attestor	a human or external sign-off	a reviewer, a domain expert, an external certification, a regulator

Table 1: The standard oracle types — the *mechanism* that produced a piece of evidence. The set is open; the guarantee a result carries is the separate **evidence_strength**.

proof; a bounded or witnessed result carries **sat**; a passing run, **tests**; an observation or a sign-off, **attested**; and an **unknown**, timed-out or errored result carries *no* strength at all. A consumer that finds a strength above the oracle’s declared capability, or any strength on an unestablished result, raises a **Defeater** residual of kind **Undermines** against the artifact (§3.5): evidence never masquerades as stronger than what was established, and the rule is enforced by the record, not by the producer’s discipline.

4.3 The evidence fingerprint and the Observation

A result is current only relative to the *subject* the oracle answered about. For a reasoner the subject is the task — the target symbol plus its dependency closure in the model; for a monitor it is the queried state — which source, which query, what it returned, as of when; for a tester it is the code under test plus the inputs; for an attestor it is the claim signed. §5 anchored freshness on a reasoner-only fingerprint of the task; the generic form is the **evidence fingerprint**: a **subject_checksum** over whatever must be unchanged for the result to still apply, a **subject_ref** naming what the subject *is* (a symbol, a query and its source, a test id, a claim id) whose disappearance is what makes evidence *Detached*, the producing oracle’s version (an advanced version obsoletes the result even on an exact checksum), and, for subjects that expire by the clock rather than by change — a fixing, a calendar year, a rate — an optional **valid_until**. The reasoner fingerprint is the reasoner profile of this record (**task_checksum** is **subject_checksum**, **target_symbol** is **subject_ref**), and a payload accepts either form. What each standard oracle type hashes is normative, so two implementations of a **monitor** agree on when an observation is stale. The derived verdict of §5.2 extends to any oracle’s evidence: a fingerprint whose subject can neither be recomputed nor re-read yields **Unknown**, which is neither fresh nor stale and which a policy may refuse for a class of claims.

A monitor’s evidence lands as an **Observation** artifact: a typed record that *a source stated this value at this time in answer to this query* — the statement in plain language, the source, the canonical query, the value, **observed_at** and an optional **valid_until**, with its attribution (strength **attested**) and its fingerprint. It anchors by **derived_from** to what it was checked *against*, so the DAG shows which established result rests on it; it discharges an **Assumption** residual by being cited from it; and it is quantifiable in goals and policies like any other artifact. This is what

lets a proof honestly rest on a premise. A proof that **settle** never books on a holiday is **Fresh** as a reasoner result for as long as the model is unchanged; the **Observation** that the calendar table matches the official calendar goes **Stale** the day the calendar is republished; and the goal that needs both shows exactly which leg moved.

4.4 Consequences for the evidence logic

Grading changes what the rest of the paper computes, in four places. A resolved acceptance item is stamped with the strength and freshness of the evidence that met it, and a goal with its **min_strength**, the weakest link it rests on (§6.4); a requirement may name the weakest grade that counts, so “tested” does not meet a requirement that asks for “proved.” Policies gain three predicates — **strength_at_least**, **oracle_type** and **produced_by** — and the declarative **reasoner** field a policy may carry desugars into **produced_by** over every evidence-bearing atom it mentions, so a policy that names an engine *fails* on a trace whose proofs came from elsewhere or carry no attribution (§7). Composition prefers the stronger of two results about the same target (§8.1). And validation checks attribution itself: an invalid strength, or an **Observation** graded above **attested**, is an error. On every rendered screen the ladder appears in plain words — proved, witnessed, tested, checked, attested (§10.3) — and the one rule the renderers may never break is that “tested” does not read as “proved.”

5 Freshness

Freshness — invariant I4 — is the soundness signal the rest of the paper leans on. It is what lets “the definition of done is met” or “this proof still holds” be *recomputed* against the current code rather than trusted from when the work was done: a result is evidence only while the thing it was computed over has not changed underneath it. This section makes that precise — the fingerprint a result carries, the derived verdict computed from it, and the formal model of that decision, proved in ImandraX.

5.1 The fingerprint

The anchor is one shared, result-kind-agnostic field. **Every** formal-reasoning result — verification, state-space, conformance, co-simulation — may carry a **reasoning_fingerprint** (Figure 6).

	task_checksum	: String	<i>hash of the target + its dependency closure</i>
	task_shape	: String [?]	<i>weaker structural hash</i>
	target_symbol	: String [?]	
reasoning_fingerprint ::=	(engine	: String [?]	)
	engine_version	: String [?]	
	model_artifact_id	: String [?]	
	model_revision	: Int [?]	

Figure 6: The **reasoning_fingerprint** anchors freshness (I4) in two parts: a strong **task_checksum** over the target *and its dependency closure* (plus a weaker **task_shape**), and the **engine**/version and **model** revision that produced the result. Freshness is then a *derived* verdict, obtained by recomputing this against the current model (Figure 7).

The critical design point is that the fingerprint hashes the *task* — the target’s dependency closure — and not the target’s own source text. Hashing source alone is unsound in *both* directions: it *misses* staleness when a result is invalidated by a change to a dependency the target’s text does not reflect, and it *over-reports* staleness on comment or formatting edits that leave the task unchanged. Closure-hashing makes a result go stale exactly when the logic it rests on changes.

5.2 The derived verdict

Freshness is a **derived verdict**, never an authored field, recomputed by comparing the stored fingerprint against the fingerprint of the *current* model (Figure 7).

$$\begin{aligned} \text{freshness} &::= \text{Fresh} \mid \text{Stale} \mid \text{Detached} \\ \text{freshness}(\varphi) &= \begin{cases} \text{Detached} & \text{if } \text{target}(\varphi) \notin M \\ \text{Fresh} & \text{if } \text{target}(\varphi) \in M \wedge \varphi.\text{task_checksum} = \varphi'.\text{task_checksum} \\ & \wedge \varphi.\text{engine_version} = \varphi'.\text{engine_version} \\ \text{Stale} & \text{otherwise (checksum mismatch, or engine advanced)} \end{cases} \end{aligned}$$

where φ is the stored fingerprint and φ' the fingerprint recomputed against the current model M . A **Detached** result is orphaned work: kept for audit, never counted as evidence.

Figure 7: The derived **freshness** verdict, computed by recomputing the fingerprint against the current model and comparing: **Fresh** when the task checksum matches (and the engine has not advanced), **Stale** on a changed task or advanced engine version, and **Detached** when the target symbol no longer exists.

A result without a fingerprint falls back to the earlier action-ordering heuristic (stale iff the target was edited at a later action). This verdict is what makes a goal’s **Property** acceptance item resolve **AcceptDone** only when its latest result is *proved*, **Fresh**, and *uncontested* (§9): a proved-but-**Stale** or **Detached** result silently reopens as a derived gap rather than continuing to read green. It is also, evaluated at a two-parent point rather than against the working tree, the exact signal the merge operator uses to compute affected sets (§8.1).

5.3 Reference freshness

The closure rule extends without change to a dependency that is not code. When a result’s closure contains a *reference artifact* (§6.5) — the model outside the repository it was judged against — the result records what it saw of that reference, its **reference_version** and **reference_checksum**. It is **Stale** once the trace’s reference artifact of that id carries a different version or checksum (the catalogue moved on) and **Detached** when the reference is no longer in the trace at all (the binding was retired). The verdict is derived per (reference, target symbol, model symbol) on the *latest* result, so re-establishing conformance against the new version heals the guard, and it surfaces as the same derived residuals a stale proof does. Two properties follow. A catalogue revision therefore stales every trace resting on the old version with no edit to any file the repository owns — exactly the case a code-fingerprint alone cannot see. And the decision is symmetric with the code side: a change to the implementation invalidates conformance through the target’s closure, a change to the reference invalidates it through the reference’s, and the record shows which leg moved.

5.4 The formal model

Freshness is not merely described; its decision core is a *specification-first* model in IML, proved by ImandraX — one of the machine-checked models of §9. We author the model directly in IML and conform the implementation to it, rather than translating the code. The core is a pure function from a fingerprint comparison to a verdict, with a worst-wins aggregate over the (possibly several) sources a result depends on (Figure 8).

```

type verdict = Fresh | Stale | Gone | Unknown    (* Gone = the paper's Detached *)

(* the per-source decision: is the file present, does the content match what was
   reasoned about, or did a logic-preserving "rescue" (e.g. a format-only edit) fire *)
let file_verdict (exists : bool) (frozen : 'h) (current : 'h) (rescue : bool) : verdict
=
  if not exists      then Gone
  else if current = frozen then Fresh
  else if rescue      then Fresh
  else                Stale

(* worst-wins over all contributing sources: gone > stale > fresh > unknown *)
let worse (a : verdict) (b : verdict) : verdict = (* ... *)
let rec combine (vs : verdict list) : verdict =
  match vs with [] -> Unknown | v :: rest -> worse v (combine rest)

(* --- invariants a silent bug would break, all proved by ImandraX --- *)

(* fresh is sound: never Fresh unless the file exists and the content matches
   (or a logic-preserving rescue justified it) *)
theorem fresh_is_sound (exists) (frozen current : 'h) (rescue) =
  file_verdict exists frozen current rescue = Fresh
  ==> exists && (current = frozen || rescue)

(* no false fresh: a Fresh aggregate hides no drifted and no missing contributor *)
theorem combine_fresh_clean (vs : verdict list) =
  combine vs = Fresh ==> not (mem Gone vs) && not (mem Stale vs)
[@@by induct ()]

```

Figure 8: The freshness decision core, authored specification-first in IML and proved by ImandraX (the *freshness lattice* model of §9, Table 3). `fresh_is_sound` forbids a spurious `Fresh`; `combine_fresh_clean` is *no-false-fresh* — the exact property the merge affected-set decision (§8.1) inherits. A third theorem, *revert-refreshes*, proves an edit-then-exact-revert returns to `Fresh`, since the verdict is a pure function of content, not of edit history.

The payoff of proving *this* core, rather than testing it, is that freshness is load-bearing everywhere downstream: a single false `Fresh` would let a stale proof read as current evidence, and every goal resolution (§6) and merge classification (§8.1) would inherit the error. *No-false-fresh* is the one property those depend on, and it is discharged once, here.

Part II

Evidence logic and composition

6 Goals

§1.2 introduced the two axes on which the evidence logic judges an agent’s work — *goals* (properties of what was established) and *policies* (constraints on how it was established) — and gave the goal half in outline: a goal is a formula over σ whose atoms range over artifacts, lineage, and freshness, resolved against the evidence rather than self-reported. This section deepens that half. We give the goal’s construction, the three orthogonal verdicts it carries, the typed acceptance criterion and the fuller property language it is a fragment of, the lineage-based resolution rule and its non-monotonic honesty overlay, and an honest account of what is authorable today versus what remains to be built.

6.1 Constructing a goal

A goal is the *definition of done*, declared once, up front, and thereafter a passive lens. It is declared through a single operation — `declare_goal` — that records five things: the `intent` (the user’s words, restated), the `scope` (the files and symbols the goal touches), optional `intent_clauses` (the intent split into individually checkable propositions), the `acceptance` items (the criteria that constitute “done”), and optional inline `policies` (the governance bar; see §7). The declared goal is embedded directly into σ as a first-class field (`goals`, canonicalizing to the empty list when no intent is declared), so that “definition of done” and “progress” are computed by the same shared resolver the rest of the substrate uses — not by a private, per-tool notion. The declared goal’s shape is shown in Figure 9.

```
goal ::= (
  goal_id      : String,
  intent       : String  (the change and why, in plain language),
  intent_author : Authorship? (who stated the intent, usually Human),
  intent_clauses : String* (intent split into checkable clauses),
  scope        : String* (files / symbols the goal touches),
  acceptance   : AcceptanceItem* (the end node: what “done” means),
  policies     : GoalPolicies? (the governance bar, §7),
  criteria_review : CriteriaReview? (the “is it right?” sign-off),
  status       : GoalStatus?,
  meta_action_id : String? (the meta-action pursuing this goal)
```

Figure 9: The `goal` type recorded by `declare_goal` and embedded in σ as a first-class field, capturing intent, scope, acceptance criteria, policies, and the criteria-review sign-off.

Two construction properties matter downstream, both consequences of the goal being decoupled from the reasoning that satisfies it. First, re-declaring is *idempotent*: re-stating a goal carries over any already-resolved criterion rather than wiping a partly-proved one. Second, because the goal is a formula over σ ’s evidence and *not* the evidence itself, it can be attached after the fact and is therefore *portable* — a team’s definition of done is a small, versioned, human-authored artifact that, once imported, re-resolves against *your* trace with no evidence transferred. What must line

up is only the referenced components: a glob scope travels cleanly, while an exact symbol reference needs that symbol present (or the durable identity of §8.2 to rebind it across a rename).

6.2 Three orthogonal axes

Where §1.2 named **met**, **governed**, and **certified** as the verdicts, here we make them precise. Trustworthy done is the conjunction of all three, and each is independent and separately reported.

Axis	Question	Mechanism	Principal
met	Does the required <i>artifact</i> exist?	criterion $\leftrightarrow$ artifact by type + lineage	the doer’s evidence
governed	Was it produced compliantly?	the goal’s policies over its cone	org / context (a pack)
certified	Is the definition of done itself <i>right</i> ?	criteria review by a non-doer	a party $\neq$ the doer

The axes are orthogonal *by construction*, and the separation is deliberate. **met** asks only whether the evidence *exists*; it says nothing about quality. A refuted **VerificationResult** still exists, so a criterion demanding a verification artifact for a component is met by it — whether that result had to be *proved* rather than left refuted is a **governed** question, decided by a policy over lineage, not a criterion field. This is why a goal can be *met but not governed*: the required artifact is present, but it was too weak (or produced non-compliantly) for the goal’s bar — a state that today reads silently as “done” and here becomes first-class and surfaced. The **certified** axis is orthogonal in a different direction: it asks not whether the formula holds but whether it is the *right* formula, and its normative rule is that the party who *meets* a goal must not be the sole party who *decides its definition of done* — so certification is a sign-off by a principal other than the doer. Because how the work was done cannot change what was established, **met** $\perp$ **governed** is a theorem we machine-check (Table 3, *governed* $\perp$ *met*).

6.3 The acceptance criterion and the property language

An acceptance criterion, in the shipping encoding, is a typed record pairing a *code component* with the *evidence artifact* that serves as its warrant (Figure 10). That is all it encodes — it does not say whether the artifact is good enough.

```

evidence_req ::= ( artifact : String  (the artifact TYPE required in the component’s lineage:)
                  :   VerificationResult | Decomp | Tests | Diff | Documentation | ... )

acceptance_criterion ::= ( id          : String,
                           statement  : String  (human “what must be true”),
                           component  : (function_ : String, file : String?),
                           evidence   : evidence_req,
                           required    : Bool,
                           covers     : String*  (intent clauses it addresses) )

```

Figure 10: The shipping acceptance-criterion encoding: a typed record binding a code component to the evidence-artifact type required as its warrant, together with the intent clauses it covers.

This fixed struct is a *concrete encoding of a fragment* of a fuller **property language** — the evidence logic proper — and every legacy form desugars into it, so no existing trace breaks. The property language expresses a goal’s “done” as a conjunction of *obligations*, each a predicate over

σ tagged with the verdict axis it feeds. The atoms are exactly the resolver predicates the implementation already computes, listed in Figure 11.

<code>has(c, T)</code>	an artifact of type T roots in component c (latest wins)
<code>proved(c)</code>	<code>has($c, \text{VerificationResult}$) $\wedge$ status $\in$ {proved, sat}</code>
<code>refuted(c)</code>	<code>has($c, \text{VerificationResult}$) $\wedge$ status = refuted</code>
<code>conforms(S)</code>	a <code>ConformanceResult</code> , complete, diverged = 0, over sources S
<code>decomposed(c)</code>	a decomposition roots in c
<code>tested(c)</code>	a region-complete / coverage-rooted suite roots in c
<code>no_open_gap(c)</code>	no open residual touches c
<code>fresh(e)</code>	evidence e is closure-fresh (§2, I4)
<code>uncontested(e)</code>	e is not contested by an open defeater

Figure 11: The atoms of the property language — the resolver predicates the implementation already computes over σ , from artifact existence and verdict checks to freshness and contestation.

Selectors turn coverage into a quantifier rather than one row per symbol: `component(f)`, `symbol(s)`, `module(p)`, `glob("payments/**")`, `tag(1)`, `scope`, and `high_stakes` (the configured high-stakes glob set). An obligation is then `[role] formula`, where `formula` combines atoms with `and`, `or`, `not`, `=>` and the quantifiers `forall  $x$  in  $sel$` / `exists  $x$  in  $sel$` , and `role` $\in$ {[met], [governed]} (default [met]). The role is not “deliverable versus standard”: a [met] obligation is an *outcome* property over the resulting artifacts (`proved( $c$ )`, `conforms( $S$ )`, `no_open_gap( $c$ )`), while a [governed] obligation is a *process* constraint over how those artifacts were produced (`formalize before verify`, `eventually reproved`). The connectives and quantifiers are shared; the *sort ranged over* — the artifact sort versus the action sort — is the goal-versus-policy distinction of §1.2. Freshness is a *modality*, not a truth condition: an obligation that holds but whose witness fails `fresh` is *true-but-at_risk*, not false — the honesty layer, made first-class in the language.

The desugaring makes one subtlety explicit. A typed {`component`, `evidence`} criterion is `has( $c, T$ )` — *existence*, so a refuted result still satisfies it — whereas the legacy `property` kind is `proved( $c$ )`, which checks the verdict and so resolves a refuted result to `blocked`. In the language one simply picks the atom that means what is intended, as the desugarings in Figure 12 show.

<code>(component:{function_f}, evidence:{artifact:T})</code>	$\rightsquigarrow$ [met] <code>has(f, T)</code>
<code>kind: property {symbol}</code>	$\rightsquigarrow$ [met] <code>proved(symbol)</code>
<code>kind: obligation {policy_id}</code>	$\rightsquigarrow$ [governed] <code>holds(policy_id)</code>
<code>kind: gap {residual_id}</code>	$\rightsquigarrow$ [met] <code>no_open_gap(target-of residual_id)</code>
<code>kind: change {symbol}</code>	$\rightsquigarrow$ [met] <code>has(symbol, Diff $\vee$ IMLModel)</code>

Figure 12: How each legacy criterion kind desugars into a role-tagged atom of the property language, so no existing trace breaks and one picks the atom that means what is intended.

6.4 Resolution by lineage

Resolution *evaluates* the formula against the trace; nothing is authored as done. A criterion `has( $c, T$ )` (and its verdict-checking cousins) resolves `met` when σ holds an artifact of type T whose lineage

roots in the component c — read from structured provenance (`target_symbol`, `derived_from`, the model’s symbol list), never from a description string. This is the fix for the “goal never ticks” defect: binding by a substring of the goal’s prose silently misses when the proved symbol’s name and the description’s wording diverge, so the rooting is computed over lineage instead. When several artifacts match, the latest by producer action is the evidence pointer. Concretely the resolver binds as shown in Figure 13.

$$\begin{aligned} \text{match}(o) &= \operatorname{argmax}_{a \in \mathcal{C}} \text{producer_action_id}(a), \quad \text{where} \\ \mathcal{C} &= \{ a \mid \text{artifact}(a) \wedge \text{type}(a) = T \wedge \text{roots_in_component}(a, c) \wedge \neg \text{contested_by_open_defeater}(a) \} \end{aligned}$$

Figure 13: The resolver’s binding rule: among artifacts of the required type whose lineage roots in the component and that are not contested by an open defeater, the latest by producer action becomes the evidence pointer.

then writes onto the *enriched copy* (never back onto the authored goal) a **status** (**done** on a match, **blocked** on a matched-but-refuted-or-contested result, **todo** on no match), the matched artifact’s id as **evidence_ref**, and the **at_risk** flag with a reason when the witness is stale or detached.

Component precision. **roots_in_component** is *target-precise*: an artifact that declares its own **target_symbol** (a verification goal, a decomposition, a targeted diff) is about *that* component, not every symbol the shared model happens to formalize. Only when nothing in the lineage names a target does resolution fall back to the model’s symbol list. Without this, a decomposition of one function would spuriously appear to root in every symbol of the enclosing model. Because the match is by component-plus-type over lineage rather than by a stored pointer, one artifact can satisfy the same obligation across *several* goals — goals are lenses over one shared evidence stream, not containers owning separate traces — and a rename is followed (**roots_in_component** accepts the source name *or* the IML symbol; §8.2).

Progress and the atom-to-helper mapping. Progress is the fraction of required items resolved: $\text{progress} = |\{\text{resolved}\}|/|\{\text{total}\}| \in [0, 1]$ (with an in-progress item counting as a half), and a goal is **met** exactly when *all* required items resolve. Each atom delegates to a shipping resolver helper — **proved/has** to the typed-lineage resolver over **roots_in_component**, **no_open_gap** to the residual scan, **fresh/uncontested** to the freshness verdict and the open-defeater check. These facts are machine-checked (Table 3, *goal resolution*): $\text{progress} \in [0, 1]$, $\text{met} \Leftrightarrow \text{all-done}$, and *at_risk never demotes*.

The at_risk honesty overlay (non-monotonic). Because the trace is a non-monotonic context (§2.2, I4), a met obligation can drift. When the evidence a met item rests on goes stale — its target’s dependency-closure content changed — the item *stays met* but is flagged **at_risk**, with a reason and a pointer to the derived stale-evidence residual. It never silently demotes to unmet, and it never silently passes as clean: the two are distinguished, and a re-established, fresh result clears the flag. A resolved obligation captures the closure checksum its witness was verified against, so an edit trips exactly the same freshness authority that governs the rest of the substrate. Figure 14 traces a met obligation through a source edit and a superseding re-proof.

Example (met but at_risk). The obligation [met] proved(refund) [fresh] is resolved at three checkpoints:

- A: refund proved fresh → done, evidence_ref = VR#7, at_risk = false
- B: source of refund edited, no re-proof → done, evidence_ref = VR#7, at_risk = true
→ (“re-verify: source drifted”; still met, never silently demotes)
- C: superseding fresh proof appends → done, evidence_ref = VR#12, at_risk = false

Figure 14: The non-monotonic `at_risk` overlay: an obligation stays met when its witness goes stale after a source edit but is flagged for re-verification, and a superseding fresh proof clears the flag — honesty without silent demotion.

6.5 Requirements against a reference

Not every requirement is a property the repository states about itself. A payment service must *implement* the PaymentIntent lifecycle its provider specifies; a reporting pipeline must implement an article of a regulation; a refactor must preserve the behaviour of the function it replaces. In each case the definition of done points at a **model outside the repository** — a catalogue entry, a venue specification, an API model, a spec-first model of the code’s own, the previous version of a function at a git ref — at a *pinned version*. The trace records that pointer as a **reference artifact**: a `RefFormalModel` whose `version` is the pinned version of the text and whose `payload.checksum` fingerprints the model’s content. A repository declares these bindings in one file (which entry, at which version, how the code must relate to it — **refinement**, **equivalence** or an **invariant** — and the weakest grade of evidence that counts); the system turns each into a reference artifact and a goal.

Conformance evidence then **names what it was judged against**. A `ConformanceResult` whose `reference_artifact_id` is a reference artifact is conformance *to the reference*, as opposed to the model↔code fidelity of §3.4; its `derived_from` includes the reference, so the reference is in the evidence’s dependency closure; its payload records the model symbol (`entry_symbol`), the code symbol (`target_symbol`), the `conformance_kind`, and what it saw of the reference at production time — `reference_version` and `reference_checksum`; and its strength says *how* conformance was established: **proof** for a refinement or equivalence theorem discharged by a reasoner, **sat** for a witness, **tests** for tests generated from the reference model’s behavioural regions and executed against the code, **static_analysis** for a schema check, **attested** for a sign-off. Validation rejects a `reference_artifact_id` that names neither an artifact nor a reference artifact.

The resolution rule of §6.4 gains one clause. An acceptance item may carry a **reference**; such an item is met only by evidence of the required type, rooted in the component, *whose reference_artifact_id equals the item’s reference*. A proof of some other property of the same component — however strong — is not conformance to the entry, and does not tick the item. Two smaller pieces complete the construction. Where the entry admits more than one reading, the reading the code implements is an **assumption** residual with `source: binding`, **waived** with the approver’s justification when a named person approved it and **open** otherwise — so an interpretation is never silent, and switching it is a change to the requirements file with an approver, not an edit an agent may make. And the policy predicate `conforms_to(r)` holds for an action that produced conformance evidence against reference *r* (an unversioned id matching any version), so “high-stakes code conforms to its entry before it ships” is one line in a pack (§7.4).

The consequence that matters most is not in the goal but in freshness. Because the reference is in the closure, the closure rule of §5 applies to it: a revision to the catalogue — a new version of

the article, a re-issued specification — stales every result judged against the old one, *without any change to the code* (§5.3). That is the sixth failure of §1.1, decided the same way as the other five.

6.6 Faithfulness — the certified axis

met and **governed** both sit *downstream* of an ungarded seam: in the real workflow the same agent that authors the definition of **done** then meets it, a textbook specification-gaming setup. An agent can be perfectly honest at resolution — never fabricate an artifact — and still “succeed” by choosing a weak or incomplete definition of **done** (the “wrong spec / vacuous proof” problem). Faithfulness cannot be decided mechanically — whether a formal acceptance faithfully captures an informal intent is the formalization gap, and intent lives in a human’s head — so the certified axis does not *verify* faithfulness; it makes the seam visible, reviewed by a different principal, coverage-checked, and hard to retrofit. Two mechanisms carry it. **Coverage:** each obligation’s **covers** field names the intent clauses it addresses, so an **intent_clauses** entry with no covering obligation surfaces as an uncovered-clause residual — completeness becomes a checkable projection rather than a judgement call. **Criteria review:** a **criteria_review** record signs off the acceptance *set* — “this is what **done** means” — and its author *must* be a party other than the doer (**Human** or **Reviewer**, never the working **Agent**). The certified verdict — **faithfulness_of** the goal — is **Approved** when a non-doer has signed off the definition *and* no intent clause is left uncovered. A criterion that *bites* is evidence of faithfulness: a property refuted with a counterexample is stronger evidence that the definition captures something real than an all-green suite of trivial edits. Certification also sees the governance overrides: a goal certified with a formal-methods pack *disabled* is a visibly weaker claim than one certified with it enforced.

6.7 Authorable today versus open

We are precise about what ships. The *leaf atoms* of the property language already exist — they are the resolver helpers that compute **proved**, **has**, **no_open_gap**, freshness, and contestation — and the shipping authorable surface is the fixed **component + evidence** struct, which is exactly the **exists-over-one-component** fragment those helpers evaluate. The lineage-precise resolution rule of §6.4, the **at_risk** overlay, and the legacy-desugaring path are all shipped (in the reference resolver, both the Python and the JavaScript implementations, held in parity by a harness). What is *not yet built* is the full property-language front-end that makes the language *authorable* in its generality:

- a *formula AST plus recursive evaluator* in place of the resolver’s fixed **kind-switch**, so **and/or/not/=>** and the role tags compose;
- a *component index* so **forall** *f* in **glob("payments/**")**, **module(p)**, and **tag(l)** can *enumerate* their sets — the one genuinely new piece of machinery, though it indexes data (per-file source symbols, the high-stakes glob set) the trace already carries;
- *stable ids for properties(S)*, so a spec’s authored verification goals are addressable for **forall p in properties(S)**.

Until then, goals are authored in the fixed struct (which desugars faithfully), and quantified coverage is expressed through **[governed]** policies rather than **[met]** selectors. The design of record is the two-sorted property language of §6.3; the evaluator for its leaves is real; the combinator and enumeration layers are the honest outstanding work.

7 Policies

Section 6 developed the *met* axis: a goal is an outcome property over the *artifact* sort, resolved against the evidence in σ . This section deepens the second axis. A goal asks *what was established*; a policy asks *how it was established*. Because both are formulas in the one two-sorted evidence logic (§1.2), a policy shares the connectives and quantifiers of a goal but ranges over the *action* sort — the finite timeline of what the agent did — and over the trace’s structural surfaces (lineage, residuals). The verdict it yields is **governed**, distinct from **met** and, as we show in §7.7, orthogonal to it. This is the *governance surface* of §2.2 made precise: governance you *run* over the trace, not a checklist or a sign-off.

7.1 The policy object

A policy is a machine-checkable rule over a trace, carrying its meaning as a *formula* rather than prose. The canonical object (POLICY_SPEC_v0_2 §6.1) is shown in Figure 15.

```

policy ::= (
  policy_id      : String,
  name           : String,
  severity       : Policy_severity,
  scope          : Policy_scope,
  kind           : Policy_kind,
  applies_when   : Policy_selector?,
  formula        : Policy_formula,
  reference_artifact_id : String?
)
```

Figure 15: The canonical **policy** record (POLICY_SPEC_v0_2 §6.1), whose **formula** field carries the rule’s authoritative machine-checkable meaning. **severity** is Info|Warning|ErrorSeverity; **scope** is Trace|Action|Artifact|Module; **kind** is TraceInvariant|ReasoningRequirement|LineageRequirement|...

The **formula** field is authoritative: if the human **description** and the **formula** disagree, the formula wins. **severity** records the consequence of failure and induces the ordering **Info** < **Warning** < **Error**; an **Error**-severity failure is a *hard* governance failure, a **Warning** a flagged gap. **scope** fixes the level at which the rule is interpreted (whole trace, an action, an artifact, or a module), and **applies_when** is an optional selector (by action type, artifact type/role, category, or path pattern) that narrows the trigger. A policy may additionally name a required **reasoner** from the reasoner registry, demanding not just *that* a claim was verified but *by what* engine — the difference between a claim *established* and one merely asserted.

A policy *attaches* to a trace in two ways. It may be materialized from a *pack* or *source* (§7.4), or it may be declared *inline on a goal* — the **policies** passed alongside **declare_goal**, evaluated over that goal’s relevance cone. Inline policies are the natural place to demand that a goal’s evidence be *good*, not merely present (e.g. “a refuted result must eventually be re-proved,” §7.6), while packs carry the shared standards a whole codebase is held to.

7.2 The formula language

The policy language (POLICY_LANGUAGE_v0_2) is **LTLf** — linear temporal logic over *finite* traces [6] — extended with *scoped-past* operators and *first-order structural predicates* over the typed

artifact graph. A trace is read as a finite sequence of actions $\tau = a_0, a_1, \dots, a_n$ over a lineage DAG, and a formula is evaluated at a position i .

Grammar. The canonical abstract syntax (POLICY_SPEC §10) is the grammar of Figure 16, written in the unicode surface ($\rightarrow, \wedge, \vee, \neg$) the gallery uses throughout.

$$\begin{aligned}
\varphi &::= \top \mid \perp \mid p \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \rightarrow \varphi \mid \varphi \leftrightarrow \varphi \\
&\quad \mid t \bowtie t \mid o_1 \varphi \mid \varphi o_2 \varphi \mid Q x \in e. \varphi \\
o_1 &::= G \mid F \mid X \mid P \mid H \mid P_{\text{target}} \mid P_{\text{chain}} \mid H_{\text{target}} \mid H_{\text{chain}} \\
o_2 &::= U \mid S \mid S_{\text{last}} \\
Q &::= \forall \mid \exists \mid \exists! \\
\bowtie &::= = \mid \neq \mid < \mid \leq \mid > \mid \geq \mid \in \\
t &::= x \mid v \mid t.f \mid \text{inputs}(t) \mid \text{outputs}(t) \mid \text{target}(t) \mid \text{ancestors}(t) \mid |e| \\
e &::= \{t_1, \dots, t_n\} \mid \{t : x \in \text{src}, \varphi\}
\end{aligned}$$

p : an atomic predicate on an action, artifact, status, or path. G, F, X, P, H : globally, finally, next, previous, historically; the **target/chain** subscripts scope a past operator to a target or its lineage chain. U, S : until, since (S_{last} : since the most recent occurrence). $t.f$: field access (e.g. $r.\text{severity}$); $|e|$: cardinality; e is a set literal or a lineage comprehension over the artifact graph.

Figure 16: The abstract syntax of the LTL_f-based policy formula language (POLICY_SPEC §10): temporal operators, first-order quantifiers, field-access terms, and set comprehensions over the typed artifact graph. This is the unicode surface used throughout the gallery.

Temporal operators. The language supports *both* future- and past-time operators over the finite timeline, plus scoped variants:

- **Future.** $G\varphi$ (globally — φ holds at every position); $F\varphi$ (finally/eventually — φ holds at some later position); $X\varphi$ (next); and the binary $\varphi U \psi$ (until — φ holds at every position until ψ does).
- **Past.** $P\varphi$ (previously — held at some earlier position); $H\varphi$ (historically — held at every earlier position); $\varphi S \psi$ (since — φ has held at every position since ψ last held); and $\varphi S_{\text{last}} \psi$ (since-last). Past operators are natural here because most trace rules look *backwards* from a triggering event, and they admit *incremental* runtime evaluation (blocking an action *before* it violates a policy).
- **Scoped past.** $P_{\text{target}}/H_{\text{target}}$ restrict the witnessing earlier position to those sharing the current action’s *target* (same file/module/artifact); $P_{\text{chain}}/H_{\text{chain}}$ restrict it to positions lying in the current action’s *derived_from lineage*. This is what distinguishes “any prior work” from “*relevant* prior work”: `research_before_edit` uses P_{target} , while `reasoning_required_for_high_stakes` uses P_{chain} .

The formal interpretation is standard finite-trace LTL: $(\tau, i) \models G\varphi$ iff $(\tau, j) \models \varphi$ for all $j \geq i$; $(\tau, i) \models F\varphi$ iff $(\tau, j) \models \varphi$ for some $j \geq i$; $(\tau, i) \models P\varphi$ iff for some $j < i$; $(\tau, i) \models H\varphi$ iff for all $j < i$; and $(\tau, i) \models \varphi S \psi$ iff there is $j < i$ with $(\tau, j) \models \psi$ and $(\tau, k) \models \varphi$ for all $j < k < i$. The scoped operators are the same, with the witness j restricted to the target- or lineage-matching positions.

Atoms, fields, and structure. At each position the atoms range over action types (`EditFile`, `GitCommit`, `Verify`, `DefineVG`, `Formalize`, `Decompose`, ...), action categories (`gateway`, `reasoning`,

activity), status predicates (`completed`, `failed`), field predicates (`rationale` $\neq \emptyset$), and path predicates (`high_stakes_path`). Structural predicates read the typed graph: artifact types (`VerificationResult`, `IMLModel`, `Decomposition`, ...), result status (`proved`, `refuted`, `sat`, `unknown`), lineage (`ancestors(derived_from)`), and set operations (`inputs`, `outputs`, `target`). Field access (`Field`) with comparisons (`cmp_op`) is what lets a policy talk about the *residual surface*: severity is *totally ordered* (`Info` $<$ `Low` $<$ `Medium` $<$ `High` $<$ `Critical`), so `r.severity` $\geq$ `High` is meaningful, while `kind` and `status` are unordered enums compared by equality, and $\emptyset$ denotes a missing value.

Each policy declares its `language_level` — `pure_temporal` (future/ past operators over atomic propositions), `scoped_temporal` (adds the scoped-past operators), or `structural` (adds quantifiers, comprehensions, and lineage predicates) — so the evaluator and the reviewer know how much machinery a check requires.

A gallery sampler. The published gallery (§7.4) is the best intuition for the language: over two hundred real policies, from CI hygiene to avionics and market governance, all in the one surface. Figure 17 shows a representative handful — each a live gallery entry, glossed by its own `description` — spanning the operator range: past-response (`P`), liveness (`F`), the since-last binary (`S_last`), the two scoped-past forms (`P_target` on the same target, `P_chain` along lineage), and negation over the connectives.

```
# tests_before_commit: every commit follows a passing test run
G(GitCommit → P(RunTests ∧ completed))

# no_secrets_committed: a commit must never introduce a secret-bearing file
G(GitCommit → ¬P(CreateFile ∧ secret_pattern))

# research_before_edit: every edit follows research on the SAME target
G(EditFile → P_target(ReadFile ∨ ReadDocumentation ∨ SearchCode ∨ AnalyzeCode))

# reasoning_required_for_high_stakes: high-stakes edits need proof in their lineage
G(EditFile ∧ high_stakes_path → P_chain(VerificationResult(proved ∨ sat) ∨
    Decomposition))

# tests_pass_before_commit: the most recent test run before each commit passed
G(GitCommit → (RunTests ∧ completed) S_last GitCommit)

# refuted_results_must_be_reproved: a refutation must eventually be re-proved
G(VerificationResult(refuted) → F(VerificationResult(proved ∨ sat)))

# no_release_without_authenticated_approval: release only after a scoped approval
G(Release ∨ Deploy → P(UserApproval ∧ authenticated ∧ approval_scope_covers))
```

Figure 17: A sampler of live gallery policies (each line commented with its policy id and description), rendered in the unicode surface of Figure 16. Operators are purple, action/artifact types blue, status predicates green, and the connectives ($\rightarrow$ $\wedge$ $\vee$ $\neg$) grey; lowercase atoms (`secret_pattern`, `high_stakes_path`, `authenticated`) are the flags a trace carries.

7.3 Evaluation semantics

Evaluating a policy φ against a trace τ computes the satisfaction relation $\tau \models \varphi$ and records a `policy_evaluation` with one of three (materially) statuses:

- **Passed** — the formula holds over the trace;
- **Failed** — it is violated, and the evaluator reports the *witnessing violating_action_ids/violating_artifact_ids*;
- **NotApplicable** — the policy’s trigger never occurs (e.g. a commit policy on a trace with no commit). **NotApplicable** results are *excluded* when a grade renormalizes compliance — a policy that does not apply neither passes nor fails a trace. (A fourth status, **UnknownEvaluation**, records the rare case where the checker cannot decide.)

Over-empty semantics. Because collections can be empty and the quantifiers are the ordinary finite-domain ones, the *vacuous* cases are load-bearing and follow the standard reading. A universal over an empty collection is *passed* ($\forall x \in \emptyset. \varphi$ is vacuously true); an existential over an empty collection is *failed* ($\exists x \in \emptyset. \varphi$ has no witness). The temporal analogue is the same: $\mathbf{G}\varphi$ over a timeline with no triggering position is vacuously satisfied — which is exactly the “vacuous case” of the response pattern in §7.6. This is why a trigger that never fires yields **NotApplicable** rather than a spurious pass: the evaluator distinguishes “held everywhere it could” from “never had the chance to fail.”

Fold-back onto σ and the governed verdict. The `policy_evaluation` records are query results (§2): they read σ and do not mutate the evidence DAG — a policy *constrains* history, it does not *define* it. The evaluations are folded back as a derived *governance verdict* (cached, never evidence), distinct from the **met** verdict of §6. A goal resolves **met** from its acceptance obligations over the artifact sort; it resolves **governed** from the policies over its cone: the inline goal policies evaluated over the goal’s relevance cone, plus the active pack’s policies. An **Error**-severity failure makes the goal *not governed* even when it is **met**; a **Warning** leaves it governed-with-a-flagged-gap. Severity thus composes with the verdict rather than the formula.

7.4 Policy sources and packs

Policies come from more than one place, and where a policy *lives* is a distribution-and-trust concern, not a semantic one: a policy’s identity and meaning are unchanged by its source. A **source** (`POLICY_SOURCES_v0_1`) is a named, typed origin — a **gallery** (a published, hash-verified static catalog over HTTP), a **local** directory of policy JSON committed in-repo, or a (future) authenticated **hub**. Sources are layered like `.gitconfig/.npmrc` (user-global then project), the order is the precedence for resolving unqualified references, and there is *no silent shadowing*: an ambiguous unqualified id across sources is an error listing the qualified options, not a coin flip.

A **pack** is a named set of policies distributed by a source and materialized together into a trace. The formal-methods pack **apply-formal-methods** has two halves: a *coverage* group (high-stakes code — money math, state machines, access control, bounds — must be backed by a proof or a decomposition in its lineage before it ships) and a *rigor* group (once the pipeline runs it must run properly: formalize before verify, refuted goals get re-proved, decompositions produce tests). Which surface counts as “high-stakes” is data-driven from the trace’s `high_stakes_paths` rather

than hard-coded. Compliance packs (e.g. DO-178C, MISRA, NIST AI RMF, ESMA/MiFID) are structurally identical — sets of the same LTLf formulas.

When a policy is materialized, the CLI stamps its *provenance* onto the trace: { **source**, **id**, **version**, **hash** }. A governed trace therefore records exactly which policies, from which sources, at which content hashes, it was checked against — closing the loop *source* $\rightarrow$ *policy* $\rightarrow$ *trace* $\rightarrow$ *sign-off* that a compliance reviewer needs. This is the *governance boundary* in operation: **ponens** *evaluates* the policies over the trace, while the producing reasoner only *emits evidence*. The party that did the work does not grade the work; the certificate travels with the claim and is re-checkable by another party (§2.2).

7.5 Compliance packs: one substrate, many regimes

The sources and packs above have one more face, and it is regulatory. The claim is not that we built a bespoke checker per standard; it is that the *same* evidence logic and governance surface — goals over the artifact sort, policies over the action sort (§1.2) — expresses obligations drawn from very different external frameworks as *policies over the same trace*. A compliance pack is a versioned set of policies mapping a named regime onto the substrate; the object being governed is always the trace, an obligation is always a policy (a temporal or lineage formula), and the verdict is always the shared Red / Amber / Green (**error** / **warning** / **clean**). Table 2 lists the packs.

Pack	External framework	What it checks (as policies)
NIST_AI_RMF	NIST AI RMF 1.0	lifecycle risk mgmt: govern / map / measure / respond
NIST_SSDF	NIST SSDF (SP 800-218)	secure-dev: release integrity, provenance, response
IOSCO_SUPERVISORY	IOSCO Supervisory Toolkit (AI)	decision logging, model versioning, oversight evidence
ESMA_MIFID_AI	ESMA statement on AI (retail)	MiFID conduct: disclosure, suitability, testing, records
MISRA_COMPLIANCE	MISRA Compliance:2020	the compliance <i>process</i> : deviations, GEP/GRP/GCS
DO_178C	RTCA DO-178C / EUROCAE ED-12C	bidirectional traceability + process objectives (by DAL)
JSF_AV_CPP	JSF Air Vehicle C++ standards	deviation approval + safety-critical obligations
CERT_SECURE_CODING	SEI CERT C / C++	risk-prioritized remediation; L1 fixed before release
CMS_AI_GUIDANCE	CMS TRA AI guidance	healthcare AI: oversight, data residency, provenance
AGENTIC_GOVERNANCE	FIX AI WG runtime governance	traffic-light runtime gating: identity, intent, telemetry
AGENTIC_EXEC_PROV	FIX AI WG six-points	decision-path reconstructability; reasoning provenance
APPLY_FORMAL_METHODS	Imandra CodeLogician / SpecLogician	high-stakes code proved/decomposed; formalize-first

Table 2: Compliance packs: the same evidence logic and governance surface instantiated against external frameworks. Each obligation is a policy over the trace; verdicts are the shared Red / Amber / Green.

Two features of the table are the point. First, the frameworks are genuinely heterogeneous — airborne-software certification (DO-178C), coding-standard process (MISRA, JSF++, CERT), AI risk management (NIST AI RMF, CMS), capital-markets supervision (IOSCO, ESMA MiFID), and

agentic runtime governance (the FIX AI packs) — yet each reduces to the two shapes the evidence logic already provides: *lineage* obligations over the artifact DAG and *ordering / presence* obligations over the action timeline. Second, the packs govern the *process around* a check, not the check itself: MISRA’s 143+ coding rules are found by a static analyzer, and the pack verifies the agent followed the deviation-and-summary process the analyzer’s output requires — turning advisory analyzer output into an *auditable* record. The substrate does not re-implement each standard’s domain checker; it supplies the one governance surface on which all of them are expressed.

Worked example: DO-178C traceability as a lineage policy. DO-178C’s defining requirement is bidirectional traceability from system requirements through high-level and low-level requirements to source code and tests. This is native to the evidence logic: the `P_chain` lineage operator walks the artifact `derived_from` DAG, so a core objective becomes a one-line policy (Figure 18).

<code>trace_code_to_llr</code>	: $G(\text{SourceCode} \rightarrow P_{\text{chain}}(\text{LLR}))$	<i>A-5; Red if no LLR ancestor</i>
<code>plan_before_dev</code>	: $G(\text{SourceCode} \rightarrow P(\text{plans_approved}))$	<i>A-1; plan before dev</i>
<code>review_with_independence</code>	: $G(\text{reviewed} \rightarrow \text{independent_review})$	<i>Amber → escalation</i>

Figure 18: DO-178C objectives as lineage and temporal policies over the trace: source-to-LLR traceability as a `P_chain` lineage obligation, plan-before-development as a past-time ordering constraint, and review-with-independence as an escalating (Amber) rather than hard-failing check.

The pattern generalizes: `IOSCO_SUPERVISORY` phrases “every AI decision is logged and its model version recorded” as a lineage obligation of the same shape; `APPLY_FORMAL_METHODS` phrases “high-stakes code is backed by a proof or decomposition before release, and counterexamples trigger a fix” as a mix of a lineage predicate (`proved / decomposed` in the component’s closure) and a temporal one (`formalize-before-verify`). One evidence logic, one governance surface, many regimes — and every verdict resolves against the trace of §2 rather than against a self-report.

7.6 Worked examples

A response (temporal) policy. The gallery policy `refuted_results_must_be_reproved` is a pure-temporal *response* pattern (Figure 19).

$$\text{refuted_results_must_be_reproved} : G(\text{VerificationResult}(\text{refuted}) \rightarrow F(\text{VerificationResult}(\text{proved} \vee \text{sat})))$$

Figure 19: The `refuted_results_must_be_reproved` policy: globally, whenever a refuted verification result appears, a proved-or-sat result must eventually follow.

Read: *globally*, whenever a refuted verification result appears, then *eventually* a proved-or-sat result appears. Walking the semantics over the finite timeline:

- $\text{Verify}(\text{refuted}) \rightarrow \text{EditFile} \rightarrow \text{Verify}(\text{proved})$: **passed**. At the position where the refuted result appears, the `F` obligation is discharged by the later `proved`.

- **Verify(*refuted*)** with no later proved/sat result in scope: **failed**. The **G** triggers, the **F** obligation is never met on the finite trace, and the evaluator reports the refuted result’s action as the witness. Note that on a finite trace **F** is not deferred to an infinite future — an obligation still open at the last position is a violation, which is what makes “fix by editing around it” insufficient: only a re-established proof closes the obligation.
- A trace with *no* refutation at all: **passed vacuously**. The **G** antecedent never fires, so every position satisfies the implication. (Depending on scope, the evaluator may instead mark this **NotApplicable** when the policy’s trigger genuinely never occurs; both agree that no violation exists.)

This is the “quality-of-evidence” check behind the governed axis: a criterion is **met** the moment a **VerificationResult** exists for its component, but the result must actually *establish* the property to count as **governed**.

A gallery reasoning policy. `async_state_transitions_verified` governs async UI state machines (Figure 20).

$$\text{async_state_transitions_verified} : \quad G(\text{Formalize} \wedge \text{async_state_machine} \rightarrow F(\text{Verify} \wedge (\text{proved} \vee \text{sat})))$$

Figure 20: The `async_state_transitions_verified` policy: a response pattern with a structural conjunct requiring every formalized async state machine to be eventually verified.

Whenever an async state machine (`idle` $\rightarrow$ `loading` $\rightarrow$ `success/error`) is formalized, its transition logic — especially error, timeout, and retry paths — must eventually be verified. It *passes* when a **Formalize** of a fetch machine is followed by a **Verify** showing all paths reach a terminal state; it *fails* for a machine where a retry-after-timeout can loop forever (a liveness violation left unverified). This is the same response shape as above, now with a structural conjunct in the antecedent.

A quantifier over the residual surface. Structural policies quantify over the trace’s declared negative space. The approval-gate policy `no_open_critical_residuals` is shown in Figure 21.

$$\text{no_open_critical_residuals} : \quad \neg (\exists r \in \text{residuals}. r.\text{severity} = \text{Critical} \wedge r.\text{status} = \text{Open})$$

Figure 21: The `no_open_critical_residuals` approval-gate policy: an existential over the residual surface asserting that no residual is both **Critical** and **Open**.

It *fails* exactly when some residual is both **Critical** and **Open**; it *passes* when the only residuals are lower-severity, or when the critical one is **Addressed**/**Waived**. The empty-collection semantics matter here: a trace with *no* residuals passes — $\exists$ over the empty surface has no witness, so its negation is true — which is the correct reading of “nothing critical is open.” The dual universal form is shown in Figure 22.

`unverified_residuals_have_suggested_check` : $\forall r \in \text{residuals}. r.\text{kind} = \text{Unverified} \rightarrow r.\text{suggested_check} \neq \emptyset$

Figure 22: The dual universal form (`unverified_residuals_have_suggested_check`): every unverified residual must carry a suggested check, turning the negative space into a work-list.

This form (`unverified_residuals_have_suggested_check`) is likewise vacuously *passed* on a residual-free trace, and turns the negative space into a work-list on any trace that declares unverified gaps. These forms compose freely under the temporal operators, e.g. the commit gate $G(\text{GitCommit} \rightarrow \forall r \in \text{residuals}. r.\text{severity} \geq \text{High} \rightarrow r.\text{status} \neq \text{Open})$.

7.7 Met versus governed

The two verdicts are **orthogonal by construction** — how the work was done cannot change what was established — because they range over different sorts of the one logic: **met** over the artifact sort (outcome), **governed** over the action sort (process). A goal can be:

- *met and governed* — the evidence exists and was produced compliantly;
- *met but not governed* — the required `VerificationResult` exists (so the criterion is **met**), but a mandatory step was skipped or the result was reached improperly: it was verified before being formalized, or a refutation was edited around rather than re-proved. Rigor is a *policy decision*, not baked into recording: a verified model with no tests and no conformance is a complete, valid artifact that resolves **met** by proof alone; “you must test/conform here” is a pack toggle over `high_stakes_paths`;
- *governed but not met* — every applicable process constraint holds, but a required outcome obligation is still open.

This orthogonality is machine-checked: Table 3 discharges *governed* $\perp$ *met* alongside the goal-resolution properties (*met* $\Leftrightarrow$ *all-done*, *at_risk never demotes*). And as with goals, composition (§8.1) re-runs the policies over the *merged* timeline: a merge does not only ask “which proofs survived,” but “was the definition of done still reached in a governed way.”

The temporal evaluator is machine-checked. The soundness of this axis is not assumed. The pure decision core of the temporal evaluator is modelled specification-first in IML and proved in ImandraX: the *policy (temporal)* model in Table 3 discharges the G/F semantics and **response-pattern soundness** — the very pattern $G(\varphi \rightarrow F\psi)$ that `refuted_results_must_be_reproved` instantiates — across 78 proof obligations, with the *verdict* $\rightarrow$ *residual* totality model ensuring every terminal evaluation state lands somewhere. So when the evaluator reports a finite-trace response policy as passed, failed, or vacuously satisfied, that judgement conforms to a proved specification: the governance surface is itself verified by the system it governs.

8 Composition

Goals and policies judge a *single* trace; *composition* is what happens when traces are combined — at a `git` merge, across agents, or across reasoners. It is the sharpest test of the whole construction, and an important technical contribution of the paper.

8.1 Composition over traces

The sharpest test of “the trace is a context” is whether contexts *compose*. The everyday instance is a `git` merge; the general instance is combining two independently-produced traces (from a hub, or from different agents or domains). A merge is the point at which two **individually-valid** results can become **jointly invalid**: branch *A* proves *f* (which calls a helper *g*); branch *B* changes *g*; the two merge with *no textual conflict and no rule broken*, yet *A*’s proof of *f* is now stale even though nothing in *A*’s diff changed. This cross-cutting invalidation is exactly what testing and review miss, and it is the reason composition must be defined, not assumed.

Principle: merge the contract, re-derive the evidence. The *goal* (the definition of done) is stable and mergeable; the *evidence* is perishable and must be recomputed against the merged code. Naively merging two evidence traces would *launder stale-green* — carry a proof forward as valid when the combination invalidated it. So evidence is never merged; it is re-established, and the merge’s job is to decide, soundly, which results need re-establishing.

The merge operator and the affected set. We add `merge`($\Gamma_{\text{base}}, \Gamma_{\text{ours}}, \Gamma_{\text{theirs}}$) as a first-class trace operation — the composition operator of the evidence logic. It records a two-parent `merge_event` (kept off the artifact DAG, so I2 acyclicity is preserved) and, for each carried-over result *R*, computes

$$\text{touched}(R) = \text{closure}(R) \cap \Delta,$$

where Δ is the merge’s component-wise change set and $\text{closure}(R)$ is *R*’s dependency closure (the sound freshness signal of I4, evaluated at a two-parent point rather than against the current source). The decision is three-way (Figure 23).

$$\text{classify}(R, \Delta) = \begin{cases} \text{CarriedForward} & \text{if } \text{touched}(R, \Delta) = \emptyset \quad (\text{closure-disjoint}) \\ \text{CarriedForward} & \text{if } \forall d \in \text{touched}(R, \Delta). \text{uninterpreted}(d) \quad (\text{contract}) \\ \text{NeedsRereasoning} & \text{otherwise} \end{cases}$$

$$\text{where } \text{touched}(R, \Delta) = \text{closure}(R) \cap \Delta.$$

Figure 23: The three-way merge classification of a carried-over result *R* against a change set Δ : carried forward when its dependency closure is disjoint from the change set, carried forward when every touched dependency was abstracted uninterpreted (a contract), and otherwise flagged for re-reasoning.

The whole operator wraps this decision in a single pass over the carried-over results (Figure 24): compute the change set once, then classify each prior result and emit its outcome, so that when the pass finishes every prior result carries exactly one verdict.

```

merge(base, ours, theirs):
  D      ← changeset(base, ours, theirs)  # per-component changes, both sides
  merged ← combine(ours, theirs)           # union of both parents' DAGs + logs
  record two-parent merge_event(base, ours, theirs) into merged
  for each carried-over result R in merged:
    touched ← closure(R) intersect D      # I4 freshness closure at merge point
    if touched is empty:                  # closure-disjoint
      emit CarriedForward(R, witness = closure(R))
    else if every d in touched is uninterpreted: # opaque contract
      emit CarriedForward(R, witness = touched)
    else:
      emit NeedsRereasoning(R, affected = touched)
  for each goal g with an unproven in-scope component: # coverage layer
    regress g's obligation
  return merged  # totality: each prior result is exactly one of
                  # CarriedForward | NeedsRereasoning | Re-reasoned

```

Figure 24: The `merge` operator in pseudocode: the change set Δ (D) is computed once, then each carried-over result is classified (Figure 23) and its outcome emitted with a witness; the coverage layer regresses a goal whose in-scope component is left unproven. On return, every prior result is exactly one of `CarriedForward`, `NeedsRereasoning`, or (after re-establishment) `re-reasoned` — the **totality** invariant machine-checked in §9.

The load-bearing property is **no-false-fresh**: a carry-forward never hides a genuinely-affected dependency. This is the freshness closure generalized, and its soundness is the *same* property proved for freshness (a fresh aggregate has no drifted contributor), so re-reasoning is bounded to the true intersection rather than the whole trace. There are two layers: the *artifact* layer (a proof’s closure drifted) and the *coverage* layer (a merge adds an in-scope but unproven component — a goal’s obligation regresses even though no existing proof changed).

Carried-forward as a positive claim. Where a residual records negative space, a `CarriedForward` artifact records a claim that was *deliberately not re-checked because it is provably safe*: it carries the closure it checked and a disjointness witness, so an auditor can *re-verify the skip* by recomputing the intersection. The merged trace’s account of each prior result is thus one of three, under a **totality** invariant — *carried forward*, *needs re-reasoning*, or *re-reasoned*: exactly one, checkably, for every prior result. This is what makes “we only re-reasoned the parts the merge affected” auditable rather than a claim to be trusted.

Opaque contracts. A dependency the proof *abstracted* is handled by its recorded abstraction kind. If R assumed g *uninterpreted* — proved its property for *all* values of g — then a merge change to g cannot invalidate R , and R carries forward (the *contract* basis above). A *pinned* dependency, or a discharged contract whose callee changed, re-reasons. The abstraction boundary is already part of the certificate (§2.2); composition just reads it.

8.2 Durable component identity

Composition is precise only if the change set Δ and the closures are computed over *components*, not text. Binding evidence to code by name breaks under a rename: a proof of `clamp` stops rooting

a goal once `clamp` becomes `clamp_int`. We give each code component a durable `component_id`, an identity layer over the descriptors an artifact already carries (per-function hashes, target symbols, contributing sources). It sits beside the record identity `artifact_id` as *which code element* versus *which record*; “the component” is the derived group over `component_id`, and corrections are append-only aliases.

Assignment resolves, most-confident-first: shared lineage $\rightarrow$ a unique content-fingerprint match $\rightarrow$ a confident similarity match $\rightarrow$ else *mint a fresh id*. The dangerous error is *conflation* (reusing an id for a different element), so the safe fallback is to mint — the mirror of never-guessing a rename. With this, a goal criterion roots *across* a rename, freshness *follows* a rename, and the merge change set matches a rename as one component rather than a delete plus an add.

8.3 Generalization: multi-reasoner composition

The evidence logic composes *claims*, *not logics*. Each object-level reasoner emits an artifact carrying a verdict and an explicit assumption boundary, mapped onto a shared verdict lattice (`proved/refuted/unknown/bounded/instance`); `ponens` conjoins results by lineage and connectives. This makes the object-logic slot pluggable: functional correctness to ImandraX, protocols and state machines to temporal logic and model checking, arithmetic to SMT, time-and-events to the event calculus. No unified super-logic is required — the intractable path — because composition happens in the evidence logic over certified claims.

The key observation is that this is *the same construction* as §8.1. A cross-branch dependency (“A’s proof used *g*, which *B* changed”) and a cross-reasoner dependency (“the arithmetic result used a fact the temporal reasoner established”) are the same object: an *explicit tracked assumption* that must be discharged, imported, or re-checked. The soundness discipline is identical — every such dependency is a recorded assumption, so no implicit assumption leakage can make a conjunction unsound — and the trace’s certificate metadata (oracle, assumptions, freshness, contested) is exactly what makes the handoff between reasoners admissible. Composition over merges and composition over logics are one mechanism; that is the deepest reason the trace-as-context coheres.

9 Formal model and verification

The invariants above are not informal. We modelled the framework *specification-first* in IML and proved it with ImandraX — the system verifying its own logic; the proofs are the conformance specification the implementation is written against. Crucially the model is not a bag of separate lemmas: it is a single **state-transition system whose state is the trace** — the machine-checked form of the trace-as-state of §2.

The machine. The model is a transition system $M = (\Sigma, \sigma_0, \rightarrow, Q)$. The **state** Σ is the trace of §2 in both its parts — an ordered *action* log and an *artifact* lineage DAG, $\sigma = (\text{actions}, \text{artifacts})$, with every artifact produced by a recorded action (events are first-class, not a detached side-list); the **initial state** σ_0 is empty; the **transitions** $\rightarrow$ are **extend** (record an action and the artifact it produces), **supersede** (retire a target’s current revision), and the two-parent **combine** (a merge); and the **queries** Q read the state without changing it — freshness, goal resolution (the **met** axis), policy evaluation (the **governed** axis), the merge affected-set classifier, and component-identity resolution. Well-formedness **wf_state** — the artifact DAG is **wf** = $I1 \wedge I2 \wedge I3$ (append-only, lineage-ordered/acyclic, evidence-grounded) *and* every artifact is grounded in a recorded action —

is proved an **inductive invariant**: it holds initially and is preserved by every transition,

$$\text{wf_state}(\sigma_0) \quad \text{and} \quad \text{wf_state}(\sigma) \wedge \sigma \rightarrow \sigma' \implies \text{wf_state}(\sigma'),$$

so *every reachable state is well-formed*. The rest of the invariants are properties of the queries over reachable states — freshness is *no-false-fresh*, the two axes are orthogonal (**governed** $\perp$ **met**), and merge’s carry-forward is sound — and of the two remaining transitions (**supersede** and **combine** also preserve **wf_state**). Everything is discharged over that one state (Table 3), not as a scattering of separate results.

```

type artifact =                                     (* a typed evidence node ... *)
  { id : int; derived_from : int list; target : int; superseded : bool;
    akind : kind; engine_backed : bool; frozen : int;
    producer_action : int }                         (* ... produced by this action *)
type action = { aid : int; atype : act; produces : int list }
type state = { actions : action list; artifacts : artifact list } (* THE STATE *)

let rec lineage_ordered (t : artifact list) : bool = (* I2: edges point at earlier ids *)
  match t with [] -> true
  | a :: r -> all_earlier a.id a.derived_from && lineage_ordered r
let rec grounded (t : artifact list) : bool =        (* I3: a VResult is engine-backed *)
  match t with [] -> true
  | a :: r -> (a.akind = VResult ==> a.engine_backed) && grounded r
let wf (t : artifact list) : bool = lineage_ordered t && grounded t
(* the full-state invariant: DAG well-formed AND every artifact has a producer action *)
let wf_state (s : state) : bool =
  wf s.artifacts && artifacts_have_producers s.artifacts s.actions

(* transitions grow actions and artifacts together *)
let extend_state (s : state) (act0 : action) (a : artifact) : state =
  { actions = s.actions @ [act0]; artifacts = s.artifacts @ [a] }
let combine_state (x : state) (y : state) : state =
  { actions = x.actions @ y.actions; artifacts = x.artifacts @ y.artifacts }
(* supersede_state retires a target's current revision (flags only; log untouched) *)

```

Figure 25: The state and its transitions, from `machine/trace_machine.iml`: the state is (**actions**, **artifacts**) — events are first-class, every artifact carries its **producer_action**; **wf_state** is the invariant; **extend_state**/**supersede_state**/**combine_state** are the transitions. Freshness, goals, policies, merge-classify and component identity are queries/decisions over this same state.

The properties are proved as ImandraX *verification goals* over the definitions above; a representative selection is shown in Figure 26.

Concern	Property proved (all over the one (actions, artifacts) state)
state (events)	wf_state : every artifact grounded in a recorded action; preserved by extend_state / supersede_state / combine_state
I1 append-only	wf preserved: extend grows, supersede flag-flips
I2 lineage / acyclic	lineage_ordered preserved; no self-reference
I3 grounded	prose is inert; grounded preserved
I4 freshness	freshness_of recomputed vs the current model; no-false-fresh
I5 reuse	never-reuse-stale; conditional growth; preserves wf
goals (met)	$\text{met} \Leftrightarrow \text{all-done}$; at_risk never demotes; $\text{progress} \in [0, 1]$
policies (governed)	$\text{LTLf } G/F$; governed $\perp$ met
merge (composition)	classify totality / no-false-fresh; combine preserves wf
component identity	never conflate ; append-only alias equivalence
verify escalation	always decides; a verdict has a witness; first-decider-wins
rename ambiguity	never guess when ambiguous
verdict totality	every terminal verdict lands somewhere ($\text{defect} \Rightarrow \text{residual}$)

Table 3: The whole framework proved as *one* state-transition machine (the trace as state): every invariant discharged over a single state — **195 proof obligations, 0 failures** (ImandraX). Composition (§8.1) and component identity (§8.2) are proved before the implementation, so the code conforms to a proved specification.

```

(* safety: wf_state holds initially and is preserved by every transition *)
theorem wf_state_empty = wf_state empty_state
theorem extend_state_preserves_wf (s : state) (act0 : action) (a : artifact) =
  wf_state s && admissible_state s act0 a ==> wf_state (extend_state s act0 a)
theorem combine_state_preserves_wf (ours : state) (theirs : state) =
  wf_state ours && wf_state theirs ==> wf_state (combine_state ours theirs)

(* I4 no-false-fresh: a fresh aggregate hides no drifted / vanished contributor *)
theorem no_false_fresh (t : trace) (present : int -> bool) (cur : int -> int) (a :
  artifact) =
  evidence_fresh t present cur && mem_art a t && is_live_result a
  ==> present a.target && a.frozen = cur a.target

(* governed _/ _ met: the two axes read disjoint projections of the state *)
theorem met_ignores_timeline (s : gstate) (g : gitem list) (tl' : timeline) =
  met_of s g = met_of { s with tl = tl' } g
theorem governed_ignores_arts (s : gstate) (trig resp : act -> bool) (arts' : trace) =
  governed_of s trig resp = governed_of { s with arts = arts' } trig resp

(* merge carry-forward is sound; a component reuse is never a guess *)
theorem merge_no_false_fresh (a : artifact) (delta : int list) (u : int -> bool) =
  classify a delta u = CarriedForward
  ==> touched a delta = [] || all_uninterp u (touched a delta)
theorem reuse_is_justified (lin : int option) (ec eid b s sid id : int) =
  resolve_component lin ec eid b s sid = ReuseId id ==>
    lin = Some id || (lin = None && ec = 1 && id = eid)
    || (lin = None && ec <> 1 && confident_sim b s && id = sid)

```

Figure 26: A representative selection of the verification goals *proved* over the machine (all discharged by ImandraX, 0 failures). **wf_empty/extend_preserves_wf/combine_preserves_wf** are the safety (inductive-invariant) goals; **no_false_fresh**, **governed $\perp$ met** (**met_ignores_timeline** + **governed_ignores_arts**), **merge_no_false_fresh**, and **reuse_is_justified** (never-conflate) are the flagship soundness goals.

Three properties are worth isolating. *no-false-fresh* (the merge affected-set) is the soundness of composition: a carry-forward provably does not hide an affected dependency. *never-conflate* (component identity) is the soundness of the rebinding that makes composition rename-robust. And *governed* $\perp$ *met* — the two axes read disjoint projections of the state, so how the work was done cannot change what was established — is proved directly on the machine. That the system can express and discharge these about *itself*, over one state, is evidence that the trace-as-context is a real logical object, not a metaphor.

Part III

The system

10 The Ponens system

The substrate is **ponens**, a reasoner-agnostic trace format and toolchain: an append-only typed trace, a residual surface, a goal/policy layer, and derived freshness, with a CLI (**trace enrich**, **trace check**, and **trace merge**). Merge is realized as described in §8.1: a report mode and a **--combine** mode that emits a materialized merged trace which the validator, enricher, and policy checker accept. The affected-set classifier reuses the closure-checksum machinery that already computes freshness, so composition is freshness “evaluated at a two-parent point.” The format spec is versioned and additive; the composition additions (the two residual kinds, the carried-forward artifact, the `component_id` field, and the merge section) were added as a backward-compatible version bump.

The concrete instantiation is a coding agent whose object logic is IML/ImandraX. Here composition is triggered on real developer workflow: a **git** post-merge hook recognizes a merge (distinct from an ordinary edit), and after the freshness recompute surfaces exactly which verified results the merge put at risk — “this merge put N verified results at risk; re-verify to restore the guarantee.” The default is to re-enrich and flag; re-verification is an explicit, bounded follow-up over the at-risk set. The whole addition is additive: a trace without composition metadata behaves exactly as before.

10.1 The CLI in practice

Every construct in this paper is exercised through one command-line tool, **ponens**, operating on a local trace file; the same verbs run inside an agent’s harness and in a reviewer’s terminal. A representative lifecycle begins by deriving a trace from an agent session, then declaring the definition of done, earning its resolution against the recorded evidence, and governing *how* the work was produced (Figure 27).

```

# Derive a trace from the latest agent session transcript (or `ponens trace init`)
ponens emit -o trace.json --summarize

# Enrich: resolve acceptance, derive the residual surface and relevance cones,
# and recompute per-result freshness (Fresh | Stale | Detached)
ponens trace enrich trace.json

# Goals: state intent + definition of done, bind an acceptance item to a symbol
ponens trace goal set      trace.json --intent "no double-charge on retry" --scope pricing.
py
ponens trace goal accept trace.json --kind property --label "idempotent charge" --symbol
charge

# Resolve the goal against the evidence (earned, not self-reported) ...
ponens trace resolve trace.json
# ... and record a non-doer's sign-off that the definition of done is RIGHT
ponens trace goal certify trace.json --by reviewer --verdict approved

# Govern: load best-practice policies and run the gate (fully offline)
ponens registry update
ponens policies add tests_before_commit --into trace.json
ponens trace check trace.json          # Passed / Failed / NotApplicable, per
policy
ponens trace report trace.json          # Markdown grade + open gaps for a PR
comment

```

Figure 27: A representative **ponens** lifecycle on a local trace file: deriving a trace from an agent session, enriching it, declaring the goal and definition of done, resolving it against the recorded evidence, and governing how the work was produced with best-practice policies.

The interchange and non-repudiation verbs (§12) sign the frozen content, bind it 1:1 to the commit under review, and project the lineage to a standard provenance vocabulary (Figure 28).

```

# Sign the content_hash with a role-bearing disposition; verify against a roster
ponens trace sign      trace.json --role auditor --disposition approved # ssh | gpg |
sigstore
ponens trace verify trace.json --allowed-signers roster

# Bind to the git commit (a Trace-Id git note) and publish for review
ponens bind && ponens push

# Project the typed-artifact lineage to W3C PROV-JSON for an external auditor
ponens trace export trace.json --to prov -o trace.prov.json

```

Figure 28: The interchange and non-repudiation verbs: signing the frozen `content_hash` with a role-bearing disposition and verifying it against a roster, binding the trace to the git commit under review, and exporting the typed-artifact lineage to W3C PROV-JSON for an external auditor.

Composition (§8.1) is a single verb. In report mode it names the verified results a merge put at risk; `--combine` materializes the merged trace, which the enricher, checker, and validator accept

unchanged (Figure 29).

```
# Merge two branch traces: carry forward the provably-unaaffected, flag the rest
ponens trace merge ours.json theirs.json --base base.json --combine -o merged.json
ponens trace enrich merged.json                # re-derive freshness at the two-parent
point
```

Figure 29: Composition as a single verb: `ponens trace merge` carries forward the provably-unaaffected results and flags the rest, and `--combine` materializes a merged trace whose freshness is then re-derived at the two-parent point.

10.2 The LTL policy checker

The governed axis of §7 is enforced by a concrete checker built into `ponens`. A policy formula — written in the unicode surface of §7 or an equivalent ASCII surface — is tokenized and parsed by a recursive-descent parser into an abstract syntax tree over the operators of the language: the future-time `G/F/X` and the binary `U`; the past-time `P/H/S/S_last`; the scoped-past `P_target/P_chain/H_target/H_chain`; the first-order quantifiers $\forall/\exists/\exists!$ over the typed artifact graph; field access and comparison; set comprehensions; and the `count` aggregate. The evaluator interprets that AST over the finite trace — the ordered action timeline — and returns a three-valued verdict per policy: `Passed`, `Failed`, or `NotApplicable` (the trigger never fired), together with *witnesses*: the action and artifact ids that support a pass or break a fail (Figure 30). `ponens trace check` runs every attached policy this way and aggregates the verdicts into the audit grade (or, for the FIX pack of §13.2, the Green/Amber/Red `GovernanceState`).

```
# a policy formula ...                parse → AST → evaluate over the action timeline
G( Release → P(UserApproval ∧ authenticated) )
```

```
$ ponens trace check trace.json
no_release_without_authenticated_approval  FAIL  (Release@a17: no prior auth
  UserApproval)
tests_before_commit                        PASS
review_with_independence                   N/A   (trigger never fired)
grade: AMBER
```

Figure 30: The LTL checker (schematic): a formula is parsed to an AST and evaluated over the action timeline, yielding `Passed/Failed/NotApplicable` per policy with witnessing ids, which `ponens trace check` aggregates into a grade.

Two things anchor the checker’s trustworthiness. First, the check is **deterministic and fully offline** — no model in the loop — which is exactly the property a *binding* governance control must have (§7; the FIX proposal’s “deterministic function over the governed state” of §13.2). Second, the language has *two evaluators kept in lockstep* — the Python CLI evaluator and a JavaScript evaluator (the browser playground and trace viewer) — checked against each other by a cross-evaluator parity harness, so a policy yields the same verdict wherever it runs. And the very same formulas *compile to IML* (over a policy-combinator library), so a policy can be handed to ImandraX; the governed

axis itself is machine-checked in the state machine of §9 (the LTL_f G/F semantics and **governed** $\perp$ **met**). The runtime checker and the formal model are two readings of one language.

10.3 The record as it is read

Everything above is computed *over* the trace; a person, and an agent between turns, reads a *summary* of it. Every screen that summary reaches — a terminal, an agent’s own prompt, a desktop, the pull-request comment of §10.4 — speaks five nouns and their states (Table 4). The formal terms of this paper stay in the record; the five words are how the record is read. What makes this more than a style guide is *where the states are computed*: once, by **ponens**, and never by a renderer. **ponens trace overview** returns one JSON with every requirement and its state, the evidence and grade behind it, every open gap with its state, the gate with the rules that block it, what to do next, the evidence per symbol, and the record’s health; **ponens trace requirements** is the requirements part alone, the join of a repository’s requirements file with the trace’s goals, reference artifacts, conformance results and freshness verdicts under seven ordered row rules; **ponens trace blame** is **git blame** for what was established — per code symbol, the strongest fresh result and the gaps anchored to it; **ponens trace next** is the record’s own to-do list, ordered fix, meet, refresh, gap, optional; and **ponens trace integrity** names the established evidence a proposed new version of the record no longer holds, so a save hook can refuse to weaken a record quietly. A renderer paints these; it derives nothing. Two consequences follow. Every surface agrees, because there is one place to disagree with. And the vocabulary is enforced: the reference renderers lint their own output for the retired terms, so “residual” or “conformance” cannot leak back onto a screen, and the one rule that must never break — “tested” does not read as “proved” — holds by construction.

Word	Absorbs	States
requirement	goal acceptance item, binding (a reference artifact and its conformance items), invariant	met · open · failed · out of date
evidence	verification, conformance, state-space analysis, tests, attestation	proved · witnessed · tested · checked · attested; fresh · out of date
gap	residual (assumption, limitation, defeater, open question, unverified), stale or detached evidence	missing · assumed · failed · out of date
record	the trace	saved · frozen · combined
gate	the policy evaluations	pass · blocked by ...

Table 4: How the record is read: the five words every rendered surface speaks, what each absorbs from the formal vocabulary, and its states. Plus *model* for what a requirement points at. The states are computed once by **ponens trace overview**; no renderer derives one.

10.4 Continuous integration: the Action

The record reaches the review it was made for through the pull request. A GitHub Action (**imandra-ai/ponens/action**) finds the single trace an agent published under **.ponens/** — or reconstructs one from a session transcript with **ponens emit** when none was published — validates it, runs the policy gate over the policies the trace carries plus any the workflow names, resolves the goals (met or open, the weakest grade each rests on, what is at risk, the open gaps), runs the requirements check of §10.3 when the repository carries a requirements file, and reports: in the

job summary, as a file, and on the pull request as *one* comment updated in place on every push. A failed error-severity policy or a blocked requirements gate fails the job under the workflow’s **fail-on**, so the check can be made required and merges blocked on it. Nothing leaves the runner and the repository is never modified; the repository dogfoods the action on itself. This is the seam at which the two review surfaces meet: the diff on the platform, the warrant beside it.

11 Trust, audit, and review

The preceding sections built the trace as an internal object: a defeasible, proof-carrying context (§2.2) over which an evidence logic resolves goals and policies (§1.2), composed at merges (§8.1) and machine-checked on itself (§9), with the system of §10 operating it. A reasoning state that only its producer can trust, however, is only half of the argument. This section is about the parties who did not do the work: the auditor who grades the record against a mechanical floor, the signer who stands behind it, the reviewer who consumes it, and the audit that re-performs it. The unifying claim is that the same certificate-carrying state that makes reasoning compose *internally* is exactly what makes it *verifiable across parties* — an auditor, a reviewing agent, or a regulator — without any of them having to trust the producer.

11.1 Audit readiness: a mechanical floor

Most “AI governance” tooling produces **attestation** — checklists and self-reported claims. The substrate takes the opposite line: a trace is **evidence**, an immutable, lineage-linked record where every “done” resolves to a specific artifact (I3) and every verdict comes from an engine an auditor can *re-run*. The difference is between “*we assert we did X*” and “*here is X, here is the proof, re-check it yourself*.” Everything in an audit-readiness assessment follows from holding that line: the trace’s value is that it is checkable, not that it is signed off.

Audit readiness is therefore not a subjective grade but a **mechanical floor** that **ponens grade** / **ponens trace check** enforce over the same evidence-logic predicates used for goals. An auditor probes a fixed set of dimensions, each of which the state answers structurally:

- **Structure / integrity.** Is the record unaltered? A canonical `content_hash` (sha256), 1:1 bound to a git commit (§12.2), answers *unaltered*; cryptographic signatures over that hash (SSH, GPG, or keyless sigstore) add *by whom*, with an optional RFC-3161 trusted timestamp for a TSA-attested *when*.
- **Rationale coverage / grounded “done.”** Is “done” real or self-reported? Acceptance resolves *deterministically from evidence* — a typed artifact in the component’s lineage — never from prose (I3).
- **Negative space.** Are the gaps disclosed? The residual surface (assumptions, unverified, out-of-scope, limitations, open questions) is first-class, and counter-evidence is a first-class **Defeater** that *blocks* a contested claim.
- **Reproducibility / verification evidence.** Can a third party re-check it? Entrypoints, environments, and re-runnable formal verdicts (**ponens trace reproduce**) mean the evidence is not merely displayed but re-derivable.
- **Lineage / freshness.** Is the evidence current? Derived freshness (**Fresh** / **Stale** / **Detached**) from a dependency-closure fingerprint (I4); a goal never resolves **done** over stale or detached evidence.

Two disciplines give the floor its teeth. The first is an ethos: **a trace with no residuals is suspicious**. An empty negative space is not evidence of completeness; a reviewer (§11.3) must treat the *absence* of declared gaps as itself reviewable, because real work leaves residue. The second is a scoping discipline that the reporting layer must never collapse: the trace separates **proved** (an engine verdict), **conforms** / fidelity-checked (model $\leftrightarrow$ code), and **certified** (a non-doer confirmed the criteria were right). More sharply, the state distinguishes what was **ESTABLISHED** (backed by a checkable artifact in lineage) from what was merely **ASSERTED** (prose, a summary, an unbacked “I tested this”). A claim of “verified” with no backing artifact is treated as **Unverified**. Because these are separable in the model and checked mechanically, audit readiness is a *floor a tool computes*, not a story a producer tells — and “proved” can never be allowed to read as “the system is correct.”

11.2 Signatures and non-repudiation

A verdict is only useful to a third party if they can trust that the trace was not altered and can see who stands behind it. A signer signs the trace’s `content_hash` with a private key or identity; anyone verifies with the public one. This does two things at once: because the signature is over the content digest, any later edit breaks it (**tamper-evidence**), and it identifies **who** signed (**non-repudiation**). Signatures live in `trace["signatures"]`, which is *excluded* from the `content_hash`, so several parties can co-sign the *same* content — the producer, a reviewer, an independent auditor — each with whatever backend they trust.

Three pluggable backends share one verification interface. `ssh` uses OpenSSH signatures (`ssh-keygen -Y`), adds no dependencies, verifies fully *offline*, and trusts a key only if it is in a git-style *allowed-signers* roster. `gpg` uses detached signatures with the signer’s public key inlined on the record (offline verification against an ephemeral keyring; trust is a fingerprint roster). `sigstore` is keyless and identity-bound: a short-lived Fulcio certificate binds the signature to an OIDC identity (email + issuer) and the proof is recorded in the public *Rekor* transparency log, so trust is “the certificate identity matches the expected issuer/subject” with no long-lived key to manage or leak. Across all three the verdict is uniform — *tampered* (the content hash changed), *invalid* (the cryptography failed), *untrusted* (a good signature from a signer not in the roster), or *valid and trusted* — and verification can be made to gate on failure (`--require-trusted`).

A sign-off is not merely a signature: it carries the reviewer’s **role** and **disposition** (e.g. `--role auditor --disposition approved`), so the attestation records *what* the party is claiming, not just that they signed. And where integrity fixes *what* was signed and the key fixes *who*, an optional RFC-3161 trusted timestamp (`--tsa`) fixes *when*: a Time-Stamping Authority countersigns the signature, giving a TSA-attested “existed by *t*” rather than a machine-clock assertion. Together these are the concrete realization of the **certified** axis (§6): a role-bearing sign-off by a named party, bound to the exact content it vouches for and to an attestable time.

11.3 Review and handoff

The consumer of a trace is increasingly not a human but a second agent: a coding agent produces the trace, a reviewing agent consumes it, and they share neither session, vendor, nor memory. Between them sits the diff, which is a *lossy* interface — it shows what changed, not what the producer tried, ruled out, verified, assumed, or left undone. The organizing move of the review model is therefore **read the trace, not the diff**: the trace’s residual surface (§11.1) becomes the spine of the review protocol, and review becomes *targeted and evidence-grounded* rather than a blind re-derivation.

The handoff contract. The producer declares its gaps; the consumer verifies the claims and hunts the undeclared ones. Concretely, the reviewer sorts the trace’s content by *checkability*: independently checkable artifacts (verification results, command/test results, decompositions, conformance results) are evidence — but only after the consequential ones are *re-checked*, because a proof travels and re-verifies; asserted prose (rationale, summaries) is *attention routing only*, never established fact; and the residual surface is the *work-list*, triaged in descending severity. The trust principle is *targeted verification, not trust*: the trace tells the reviewer *where to look* and *what was claimed*; the reviewer establishes *what is true*. The reviewer’s distinctive job — the reason independent review exists — is Step 4: anything the change touches that is neither verified nor declared is an *undeclared* residual that the reviewer must raise. The declared surface does not replace this hunt; it makes it *tractable* by shrinking the space to discover from scratch.

Review items and verdicts. The reviewer writes back into the collaboration surface as `review_items` (each carrying its `target` and a `blocking` flag), comments, and residual status updates (`ResidualAcknowledged` for accepted gaps, `ResidualWaived` for explicit, auditable acceptance). The overall disposition is one of `approve`, `request-changes`, or `escalate-to-human`. The disposition rule is exacting: *approve only when no open residual is blocking, every consequential checkable claim has been re-verified, and the undeclared-gap hunt surfaced no high-severity gap*. Open questions are routed to a human by construction — they are decisions, not checks, and an agent must not auto-approve over one. Above individual traces sits the *review case*: a problem-centered workflow container that accumulates verdicts across a chain of traces and carries the governing invariant that all current work should retrace to the original problem statement while every step stays within policy.

Immutability closes gaps forward. A reviewer never mutates a trace to “fix” a gap. Traces are append-only (I1), so a gap is closed in a *successor trace* that **supersedes** the original: the reviewer emits a successor-trace request naming the `residual_ids` that must close, and the fix produces a new, immutable record. The supersedes chain is thus the audit trail of the review itself — what was contested, what was requested, and where it was discharged — rather than a mutable document whose history is lost.

11.4 The audit process

The pieces above compose into an audit that is *re-performed*, not merely read. Given a commit bound to a trace (§12.2 — a `Trace-Id` and a git note anchoring the frozen snapshot), an auditor proceeds:

1. **Verify signatures** (§11.2): confirm the snapshot is untampered (the signature is over the `content_hash`) and that each sign-off comes from an identity in the trusted roster. This establishes *what* is being audited and *who* stands behind it before any claim is examined.
2. **Re-derive the evidence.** Because every verdict is engine-grounded and every result carries a dependency-closure fingerprint (§3), the auditor re-runs the proofs and re-computes freshness rather than taking the recorded verdicts on faith — the claims are reproducible, not asserted. Carried-forward skips introduced by a merge (§8.1) are re-checked by recomputing the closure/change-set disjointness witness, so even the work a merge deliberately did *not* re-run is independently verifiable.

3. **Re-resolve goals and re-evaluate policies** (§6, §7): the **met** and **governed** verdicts are recomputed against the trace, so “done” and “done in a compliant way” are checked, not trusted.
4. **Work the residual surface.** The declared negative space — assumptions, limitations, defeaters, open questions — is where an auditor’s attention goes first; its *emptiness* is itself a warning sign (§11.1), since honest reasoning about a real system almost always leaves something open.
5. **Issue a verdict** (§11.3): approve, request changes, or escalate, recorded as a review case. Because the trace is immutable, remediation does not overwrite the record — it lands in a successor trace, and the auditor can re-sign the content they approved.

The defining property is that authority derives from *reproducible proof, not reputation*. An auditor need not trust the producer — or the model that produced the work — because the warrant travels with each claim and can be re-checked, while the signature fixes accountability. This is precisely what makes the substrate suitable for a verification role that is *independent* of the party that did the work: the audit is a re-computation over a signed, immutable object, not a reading of someone’s report about it.

Taken together — a mechanical audit floor, role-bearing signatures over the content hash, a residual-driven review handoff, and an audit that is re-performed rather than read — these are the sense in which the trace is not merely recorded but *verifiable by a party other than the one that produced it*. That is the same warrant-travels-with-the-claim property that made composition sound (§8.1); here it is turned outward. §12 completes the picture with how the record leaves the system and how it stays bound to the code it explains.

12 Interchange and sync

Two more outward faces remain, both about *transport* rather than trust: how the record is read by tooling that was not built for it, and how it travels with the code it explains. Both are deliberately narrow. A provenance projection carries lineage and attribution and nothing else; a git binding carries the trace’s identity and nothing else. What the record *means* — freshness, contested-ness, the residual surface, the verdicts — stays with **ponens**, where it can be recomputed.

12.1 Interchange: projecting to W3C PROV

An agent’s provenance ought to be legible to tools that were not built for our format. We therefore project σ onto **W3C PROV** [2], the standard vocabulary for provenance (**ponens trace export** `--to prov`, emitting PROV-JSON). The mapping is close to structural: a typed artifact becomes a **prov:Entity**, an action a **prov:Activity**, the assistant (with its model) a **prov:Agent**, and the trace itself a **prov:Bundle**. The lineage edges carry over one-to-one — **producer_action_id** to **wasGeneratedBy**, an action’s inputs to **used**, **derived_from** to **wasDerivedFrom**, and **supersedes** to a **wasDerivedFrom** of type **prov:Revision** (i.e. **wasRevisionOf**). Every artifact and action is **wasAttributedTo** / **wasAssociatedWith** the agent. Edges whose endpoints are not present in the trace (a dangling **derived_from**) are dropped rather than emitted, so the result is a well-formed PROV instance.

The essential point — and the reason this belongs beside §2.2 rather than in place of it — is that *PROV is an interchange view, not a reasoning state*. PROV models *what was derived from*

what, by whom; it has no vocabulary for the evidence-logic certificate that makes the trace a context in the first place. Concretely, the projection is **faithful for lineage and attribution** and **lossy for judgement**:

- **Freshness** (I4) is *derived* by **ponens** from a dependency-closure fingerprint; PROV's **wasInvalidatedBy** records an invalidation *event someone asserts*, not a *computed* staleness, so it is not emitted.
- **Residuals and defeaters** — the negative space and the contested-ness of a claim — have no PROV relation. A **Residual** is exported as a **prov:Entity** with its **kind**, **severity**, and **status** riding as **ponens:** attributes, but PROV assigns them *no meaning*; the blocking of a contested claim by counter-evidence stays a **ponens**-side computation.
- **Verification verdicts** (proved / refuted / unknown) ride as an opaque **ponens:verdict** attribute on the entity.
- **Policies** — the temporal-logic governance axis — have no analogue; PROV-CONSTRAINTS is a fixed consistency checker, not a policy language.

The rule of thumb is *lineage and attribution round-trip; judgement does not*. Export is deliberately one-way: a consumer that needs the certificate — freshness, contested-ness, the residual surface, the policy verdict — reads the **ponens** trace, and the standard vocabulary is offered so that provenance tooling can at least consume chain-of-custody and derivation without learning our format. This is the same distinction the paper draws throughout: a log records *what happened*; a context carries *the warrant*. PROV is the log projection of a state that is more than a log.

12.2 Sync and binding

For the warrant to be *useful* to a downstream party, it must travel with the code it explains. Three facts live in three places and no single layer owns all three: the **local trace file** is canonical for *content* (immutable, content-hashed); the **git commit** is canonical for *version* (which code state the reasoning is about); and a **hub** is canonical for *decisions* (status, comments, review items, snapshots, policy runs). The CLI's job is to bind them and keep them from silently diverging — in one line, it makes the hub a git remote for reasoning.

ponens **bind** is the load-bearing verb. It stamps the trace with **repo / branch / commit_sha** from git, and writes the trace identity *back* onto the commit — as a **Trace-Id** trailer or, by default, a **git** note (**refs/notes/ponens**, non-mutating). This makes the relationship resolvable in both directions, **commit_sha** $\Leftrightarrow$ **trace_id**, so the reasoning is discoverable from the commit (e.g. in a pull-request view) and vice versa. The binding is **1:1**: a trace is bound to exactly one commit, and iteration produces a *new* commit and a *successor* trace rather than overwriting the old one — the same immutability discipline as §11.3, now mirrored one-to-one with git history. **push** / **pull** move the trace and the hub's decision state (**push** is idempotent and creates a successor on a content change; **pull** is read-only with respect to the working tree), and **status** names the one steady state (**syncd**) against the divergences (**stale**, **dirty**, **behind**) that each map to a concrete next command.

The design decision is **binding-only**: git carries the trace and the link, and *only* those. This is what lets the warrant travel with the code *without* committing the churning raw trace state or reinventing review inside git — git notes do not merge cleanly and committed review files conflict on every parallel comment (this is the local regime the merge composition of §8.1 operates within:

git merges the code and the binding; `ponens` re-derives the evidence). Human review of the *diff* stays on the git platform; computable governance and the frozen, signed *snapshot* (the portable audit artifact with no platform equivalent) stay on the hub; the commit is the shared anchor connecting them. Let the platform own diff review, let the hub own governance over the reasoning, and connect them through the commit — do not reimplement either inside the other.

Part IV

Demonstration and discussion

13 Worked examples

The construction is easiest to see end-to-end. We walk two scenarios — one from software development, one from agentic trading — each exercising the full machinery: a goal, its resolution against evidence, governance by policy, and (for the first) composition across a merge. The two differ only in their *object logic* and *policy vocabulary*; the evidence logic over the trace is identical. The trading example runs the FIX Trading Community’s runtime-governance pack [13] verbatim.

13.1 A verified pricing change through a merge

An agent is asked to make a payment function `charge` idempotent under retries: a retried request must never double-charge. This pays off motivating example 1 (§1.1) concretely.

1. **Declare the goal.** The definition of done is stated and bound to the component it constrains (§6):

```
ponens trace goal set trace.json --intent "charge is idempotent
under retry" --scope pricing.py
ponens trace goal accept trace.json --kind property --label "no
double-charge on retry" --symbol charge
```

The goal is now a first-class field of σ ; its acceptance item names the property and the component `charge`, not a line of prose.

2. **Establish the evidence.** The agent formalizes `charge` in IML, defines a verification goal, and ImandraX discharges it. A `VerificationResultArtifact vr1` with status `proved` enters the lineage, carrying a `reasoning_fingerprint` over `charge` and its dependency closure — which includes the helper `apply_discount` (§3).
3. **Resolve and govern.** `ponens trace resolve` walks the lineage: an artifact of the required type roots in `charge`, is `Fresh`, and is uncontested, so the acceptance item resolves `met` and the goal is met — earned from evidence, not self-reported (§6). `ponens trace check` runs the governance pack (e.g. tests-before-commit) and returns Green (§7); `ponens trace sign` and `ponens bind` freeze and attribute the record (§11.2).
4. **A change arrives on another branch.** A colleague, on a separate branch, refactors `apply_discount` — a function in `vr1`’s dependency closure — to change rounding. Their branch’s own trace is internally consistent and never mentions `charge`; in isolation, each side is valid.

5. **The merge, decided soundly.** A git merge fires the post-merge hook, which runs `ponens trace merge ours.json theirs.json --base base.json --combine` (Figure 29). The classifier intersects `vr1`’s dependency closure with the merge’s change set: `apply_discount` is touched and was not abstracted `uninterpreted`, so `vr1` is *not* carried forward — it becomes a `NeedsRereasoning` residual and the goal’s verdict drops to `at_risk` (§8.1). The report reads: “this merge put 1 verified result at risk; re-verify to restore the guarantee.” By the *no-false-fresh* property (proved, §9), the proof is never silently kept.
6. **Close the loop.** Re-running the proof over the at-risk set either restores `proved` (the goal returns to `met`, freshness `Fresh`) or produces a counterexample — a `Defeater` that `Rebuts vr1`, making the claim contested and resolving the acceptance item `AcceptBlocked` (§6). Either way the result is a checkable verdict, not a silent regression: the individually-valid proof and the individually-valid refactor were *jointly* invalid, and the trace made that decidable.

13.2 Governing an agentic trading workflow (FIX)

An execution agent works an order in secondary markets. Its run is recorded as a trace whose actions use the agentic vocabulary the FIX AI Working Group’s runtime-governance proposal [9] assumes — `Retrieve` (market data), `Compute` (order size), `Draft` (the order), `Release` (send to venue) — each carrying the governance predicates the runtime emits per action (identity resolved, vLEI present, intent resolved, tool allowlisted, provenance and recency checked, deterministic recompute, authenticated approval). We govern this trace with the FIX Trading Community’s *Agentic AI Runtime Governance* pack [13], which maps the proposal’s traffic-light (Green / Amber / Red) scheme onto `ponens` policies (Figure 31); the underlying capability model and four-tier risk scheme are from Szpruch et al. [10]. The same shape appears in their independently-developed credit-memo example — a labelled transition system with a guard on every transition (retrieval recency, deterministic recomputation, an authenticated-approval gate before release). Read in our terms, that trajectory is a trace and those guards are policies of the form in Figure 31, which `ponens trace check` evaluates as the deterministic governance function both lines of work call for.

```

no_release_without_authenticated_approval : Red; approval & release gating
  G(Release → P(UserApproval ∧ authenticated ∧ approval_scope_covers))
dual_approval_critical                     : Red; Tier 4, two approvers
  G(Release → P(UserApproval ∧ approver_1) ∧ P(UserApproval ∧ approver_2))
retrieved_data_attributed                  : Red; Szpruch C1 (retrieval)
  G(Retrieve → provenance_checked ∧ recency_checked)
numeric_recomputed_deterministically       : Red; Szpruch C2 (numerics)
  G(Compute → deterministic_recompute)
credential_not_expiring                    : Amber; identity & auth
  G(action → ¬ credential_expiring)

```

Figure 31: A slice of the FIX Trading Community runtime-governance pack [13, 9]: traffic-light conditions expressed as temporal-logic policies over the trading agent’s execution trace. `ponens trace check` aggregates the per-policy verdicts into a single Green / Amber / Red `GovernanceState`.

1. **Attach and check.** `ponens trace check trace.json` evaluates each traffic-light condition as a policy over the trace and aggregates the verdicts: all-pass is Green, a warning-severity

failure is Amber, an error-severity failure is Red (§7).

2. **A red flag.** Suppose the agent issued a `Release` with no prior authenticated, scope-covering approval. `no_release_without_authenticated_approval` (Figure 31) fails at **error** severity, so the aggregate is **RED**: `ponens trace check` exits non-zero and names the failed policy — the `GovernanceFlags` the proposal wants carried in a new `FIX` field. In parallel a credential nearing expiry trips `credential_not_expiring` at **warning** severity (**Amber**), which is reported but never collapsed into the Red.
3. **Deterministic, model-independent enforcement.** The verdict is a deterministic function over the trace, executing independently of the language model — exactly the property the proposal requires of any *binding* (as opposed to advisory) control [9, §10.3]: “a governance decision that cannot be expressed as a deterministic function over the governed state ... is advisory guidance, not binding enforcement.” A `ponens` policy *is* that function.
4. **Remediate and re-gate.** A Tier-4 (Critical Autonomous) deployment additionally requires `dual_approval_critical`. Once the agent obtains authenticated dual approval before `Release`, `ponens trace check` returns Green; the signed, bound trace (§12) is the auditable `GovernanceState` record — verifiable by the venue or a regulator without trusting the agent’s operator.

In both examples the same object — a proof-carrying trace queried by the evidence logic — carries the verdict; only the object logic (IML/ImandraX proofs versus governance predicates over a trading trajectory) and the policy vocabulary differ. That is the multi-reasoner thesis of §8.3 in miniature.

14 Related work

Provenance. W3C PROV [2] standardizes *how* an artifact came to be, for interchange; it is a lossy view of, not a substitute for, a reasoning state — it carries neither the evidence-logic certificate (freshness, contested-ness) nor a notion of composition. *Non-monotonic and defeasible reasoning.* The trace is a defeasible context in Pollock’s sense [3]: defeaters record counter-evidence and retract conclusions; the query-carrying property connects it to answer-set programming [4] and the event calculus [5]. *Proof-carrying code* [7]: claims travel with certificates checkable by the consumer; we extend the idea from a single artifact to a composable context. *Truth-maintenance systems* [8]: dependency-directed retraction is the local analogue of our non-monotonic, well-founded context — the trace can be seen as a proof-carrying, append-only TMS over a codebase, with merges as the composition operation TMSs do not define. *Version control.* Git infers renames heuristically at diff time and merges text without any notion of semantic invalidation; §8.2 gives the durable identity git omits, and §8.1 the semantic merge it does not attempt. *Software supply-chain attestation.* in-toto [11] and SLSA [12] sign statements about how an artifact was built — the steps, their materials and products — and verify a chain of them against a layout; they are the nearest prior art to the signature and audit surfaces of §11. An attestation records that a step ran; a trace records what the step *established*, how strongly, on what it depends and what it left open, and the same signature discipline (§11.2) applies over that richer statement.

Runtime governance for agentic AI. A parallel line in financial Model Risk Management, developed independently of this work, arrives at strikingly similar premises: agentic risk is a property of *execution trajectories* rather than of an input–output mapping, and binding governance must be a *deterministic function over the governed state*, executed independently of the language model [10, 9].

That is the object and the discipline we arrive at here as well — their governed trajectory is our trace, and their governance function is a **ponens** policy (§7) that **ponens trace check** evaluates — and we read the convergence as mutual corroboration rather than either deriving from the other. Ponens itself was driven by industrial applications in capital markets — certifying counterparty connectivity against venue FIX specifications, and governing agentic trading workflows — rather than by a governance theory; the convergence is therefore between field practice and an independently-derived MRM programme, which we take as the stronger evidence that treating the execution trajectory as the governed object is the right move. The agreement runs deeper on two points that our construction independently makes concrete. Both hold that verification must be *decomposed by claim structure* — logical, numeric, retrieval, consistency — rather than delegated to one semantic judge, which here is multi-reasoner composition (§8.3); and both hold that an LLM-as-judge conflates *plausibility* with *correctness*, which here is the ESTABLISHED-versus-ASSERTED discipline (§11.1). The two frameworks are complementary in scope: theirs is a capability-catalogue MRM *programme* for a bank; ours contributes a concrete trace format, a policy language and its formal semantics, the goal axis, *composition over traces at a merge* — a notion their setting, concerned with single-run orchestration and change control, does not address — and machine-checked foundations that make the governance function checkable and its warrant portable across parties.

15 Conclusion

Treating the reasoning trace as a state — a defeasible, proof-carrying context rather than a log — is not bookkeeping; it is what makes agent reasoning *compose*. The same append-only structure serves as memory, governance surface, and logical context, and the certificate every claim carries is exactly what a merge (or a second reasoner) needs to decide, soundly, what a change preserves and what it invalidates. We gave the model, its composition operator, and a durable identity for binding evidence to code across change; we proved the load-bearing invariants by the system on itself; and we showed the construction generalizes from merges to multiple logics. The result is a substrate on which an agent’s work is not merely recorded but *verifiable* — and, because the warrant travels with the claim, verifiable by a party other than the one that produced it.

Availability. **ponens** is open source. The toolchain and the full trace, goal, and policy specifications are at <https://www.ponens.dev>; the source is at <https://github.com/imandra-ai/ponens>.

References

- [1] Passmore, G. et al. *The Imandra Automated Reasoning System*. IJCAR, 2020.
- [2] Moreau, L., Missier, P. (eds.). *PROV-DM: The PROV Data Model*. W3C Recommendation, 2013.
- [3] Pollock, J. *Defeasible Reasoning*. Cognitive Science 11(4), 1987.
- [4] Gelfond, M., Lifschitz, V. *The Stable Model Semantics for Logic Programming*. ICLP, 1988.
- [5] Kowalski, R., Sergot, M. *A Logic-Based Calculus of Events*. New Generation Computing 4, 1986.
- [6] Pnueli, A. *The Temporal Logic of Programs*. FOCS, 1977.

- [7] Necula, G. *Proof-Carrying Code*. POPL, 1997.
- [8] de Kleer, J. *An Assumption-Based TMS*. Artificial Intelligence 28(2), 1986.
- [9] FIX Trading Community, AI Working Group. *Proposal on Agentic AI Runtime Governance under FIX Protocol Extensions*. R. Healey & K. Houston, 12 Jun 2026.
- [10] Szpruch, L., Sudjianto, A., Bhatti, T., Ang, G. *Scalable Runtime Governance for Agentic AI in Financial Services*. SSRN 6567199, 2026.
- [11] Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., Cappos, J. *in-toto: Providing farm-to-table guarantees for bits and bytes*. USENIX Security, 2019.
- [12] OpenSSF. *Supply-chain Levels for Software Artifacts (SLSA), version 1.0*. <https://slsa.dev>, 2023.
- [13] FIX Community. *Agentic AI Runtime Governance — ponens policy pack*. <https://www.ponens.dev/organizations/fix-community>.